



الجمهورية العربية السورية

وزارة التعليم العالي والبحث العلمي

الجامعة الوطنية الخاصة

كلية الهندسة - قسم هندسة الحاسوب

مشروع أعد لنيل درجة الإجازة في هندسة الحاسوب بعنوان:

تصميم وتنفيذ أداة API و MCP لتحويل المستندات من

PDF إلى Markdown

إعداد الطالب:

عبد الله محمد نوار الشقفة

إشراف الأستاذ الدكتور:

إبراهيم الشامي

العام الدراسي 2025 - 2026

فهرس المحتويات

1	الفصل الأول: المقدمة.....
2	1-1 تمهيد
2	2-1 المشكلة البحثية وفكرة المشروع
3	3-1 أهداف المشروع.....
4	4-1 أهمية المشروع.....
5	5-1 الفئة المستهدفة والمستفيدين
6	2 الفصل الثاني: الأسس النظرية والتقنيات المعتمدة.....
7	1-2 المفاهيم النظرية الأساسية
7	1-1-2 لغات التوصيف الخفيفة ومنهجية الوثائق كنصوص برمجية (Docs-as-Code)
9	2-1-2 الحوسبة بدون خوادم والحوسبة الطرفية (Serverless & Edge Computing)
11	3-1-2 عرض المتصفحات الرأسية (Headless Browser Rendering)
13	4-1-2 تخطيط النصوص ثنائية الاتجاه (Bidirectional Text - BiDi).....
15	5-1-2 الرسوميات المتجهة مقابل الصور النقطية في المستندات (Vector vs. Raster Graphics)
18	2-2 التقنيات، الأدوات، وإطارات العمل المستخدمة.....
18	1-2-2 البنية التحتية السحابية (Cloud Infrastructure)
21	2-2-2 الخلفية وإطار عمل الويب (Backend & Web Framework).....
23	3-2-2 محرك التحويل ومعالجة المحتوى (Conversion Engine & Content Processing)
25	4-2-2 الواجهة الأمامية والتصميم البصري (Frontend & Visual Design)

27.....	3-2 الأعمال السابقة والدراسات المرتبطة
28.....	1-3-2 أدوات وتقنيات معالجة المستندات الرقمية (Digital Document Processing Tools)
28.....	2-3-2 البنى السحابية الحديثة لمعالجة المستندات (Cloud-Based Document Processing)
28.....	3-3-2 الدراسات الأكاديمية المرتبطة (Related Academic Studies)
29.....	4-3-2 التحليل النقدي وتحديد الفجوة البحثية (Critical Analysis & Research Gap)
31.....	3 الفصل الثالث: تحليل وتصميم النظام
32.....	1-3 تحليل النظام
32.....	1-1-3 تحديد الجهات الفاعلة (Actors)
33.....	1-1-3 مخطط حالات الاستخدام (Use Case Diagram)
35.....	2-3 تصميم قاعدة البيانات (Database Design)
35.....	1-2-3 الكيانات الرئيسية (Main Entities)
36.....	2-2-3 العلاقات بين الجداول (Relationships)
37.....	3-2-3 تصميم مخطط الكيانات والعلاقات (Entity Relationship Diagram)
38.....	4-2-3 خصائص التصميم
38.....	3-3 التصميم المعماري للنظام (System Architecture)
39.....	1-3-3 طبقة العميل (Client Layer)
39.....	2-3-3 طبقة الواجهة الخلفية (Application Layer)
40.....	3-3-3 محرك التحويل (Conversion Engine)
40.....	4-3-3 الخدمات السحابية والخارجية (Cloud & External Services)
41.....	5-3-3 تدفق البيانات بين طبقات النظام

42.....	6-3-3 مزايا التصميم المعماري
43.....	4-3 سير عمل النظام (System Workflow)
43.....	1-4-3 استقبال الطلب والتحقق من صحته
44.....	2-4-3 معالجة محتوى المستند
44.....	3-4-3 معالجة المخططات البيانية
45.....	4-4-3 إنشاء ملف PDF
45.....	5-4-3 تسجيل العملية وإرجاع النتيجة
48.....	4 الفصل الرابع: تنفيذ النظام والتطبيق العملي
49.....	1-1-4 الصفحة الرئيسية للنظام
50.....	2-1-4 تسجيل الدخول وإنشاء الحساب
51.....	2-4 واجهة تحويل Markdown إلى PDF
52.....	1-2-4 إدخال النصوص والمعاينة المباشرة
53.....	2-2-4 رفع ملفات Markdown
54.....	3-2-4 تنفيذ عملية التحويل
55.....	3-4 دليل استخدام Markdown
56.....	4-4 توثيق واجهة برمجة التطبيقات (API Documentation)
56.....	1-4-4 الصفحة الرئيسية لتوثيق API
58.....	2-4-4 واجهات المصادقة (Authentication APIs)
59.....	3-4-4 دعم تعدد اللغات
60.....	4-4-4 دعم OpenClaw Skill

61.....	Model Context Protocol (MCP) دعم بروتوكول
63.....	MCP أدوات خادم 1-5-4
64.....	6-4 نتائج التطبيق واختبار وظائف النظام
64.....	1-6-4 التحقق من خصائص النظام
65.....	2-6-4 اختبار التسيقات الأساسية
66.....	3-6-4 اختبار دعم المعادلات الرياضية (LaTeX)
67.....	4-6-4 اختبار توليد النصوص البرمجية
68.....	5-6-4 PlantUML مخططات
71.....	5 الفصل الخامس: الخاتمة والآفاق المستقبلية

فهرس الأشكال

- الشكل 2-1: سير العمل في منهجية الوثائق كنصوص برمجية (Docs-as-Code Workflow) 9
- الشكل 2-2: مقارنة بين الحوسبة السحابية (Cloud Computing) والحوسبة الطرفية (Edge Computing) 11
- الشكل 2-3: مقارنة بين اتجاهي الكتابة من اليسار إلى اليمين (LTR) ومن اليمين إلى اليسار (RTL) 15
- الشكل 2-4: مقارنة بين الرسوميات النقطية (Raster Graphics) والرسوميات المتجهة (Vector Graphics) 17
- الشكل 2-5: معمارية البنية التحتية السحابية المعتمدة على Cloudflare 20
- الشكل 2-6: دورة معالجة طلبات واجهة برمجة التطبيقات في الواجهة الخلفية 23
- الشكل 2-7: تطور البنى المعمارية للأنظمة البرمجية وصولاً إلى المعماريات الحديثة 29
- الشكل 3-1: مخطط حالات الاستخدام (Use Case Diagram) 34
- الشكل 3-2: مخطط الكيانات والعلاقات (Entity Relationship Diagram) 37
- الشكل 3-3: التصميم المعماري للنظام (System Architecture) 42
- الشكل 3-4: مخطط سير عمل النظام لتحويل Markdown إلى PDF 46
- الشكل 4-1: الواجهة الرئيسية لنظام تحويل Markdown إلى PDF 50
- الشكل 4-2: نافذة تسجيل الدخول وإنشاء الحساب 51
- الشكل 4-3: واجهة إدخال Markdown والمعاينة المباشرة 53
- الشكل 4-4: واجهة رفع ملفات Markdown 54
- الشكل 4-5: تنفيذ عملية تحويل Markdown إلى PDF 55
- الشكل 4-6: دليل Markdown السريع 56
- الشكل 4-7: الصفحة الرئيسية لتوثيق واجهة برمجة التطبيقات 57
- الشكل 4-8: الميزات المدعومة في النظام 58
- الشكل 4-9: توثيق واجهات التسجيل وتسجيل الدخول 59
- الشكل 4-10: واجهة دعم تعدد اللغات 60

61.....	الشكل 4-11: توثيق OpenClaw Skill
62.....	الشكل 4-12: الواجهة الرئيسية لخادم MCP
63.....	الشكل 4-13: أدوات خادم MCP
65.....	الشكل 4-14: التحقق من خصائص النظام المدعومة
66.....	الشكل 4-15: اختبار التنسيقات الأساسية
67.....	الشكل 4-16: اختبار دعم المعادلات الرياضية
68.....	الشكل 4-17: اختبار تلوين النصوص البرمجية
69.....	الشكل 4-18: اختبار المخطط التسلسلي (Sequence Diagram)
69.....	الشكل 4-19: اختبار مخطط الفئات (Class Diagram)
70.....	الشكل 4-20: اختبار مخطط التدفق (Activity / Flow Diagram)

فهرس الجدول

- الجدول 2-1: مقارنة بين أنظمة كتابة وتنسيق المستندات 7
- الجدول 2-2: مقارنة مسؤوليات المطور بين نماذج الحوسبة السحابية المختلفة 10
- الجدول 2-3: مقارنة بين المتصفحات التقليدية والمتصفحات الرأسية 12
- الجدول 2-4: تصنيف المحارف في خوارزمية Unicode للنصوص ثنائية الاتجاه 14
- الجدول 2-5: مقارنة بين الرسوميات النقطية والرسوميات المتجهة 16
- الجدول 2-6: مكونات البنية التحتية السحابية المستخدمة في المشروع 19
- الجدول 2-7: أدوات تطوير الواجهة الخلفية وإطارات العمل المستخدمة 22
- الجدول 2-8: مكونات محرك التحويل ومعالجة المحتوى 24
- الجدول 2-9: تقنيات الواجهة الأمامية والتصميم البصري 26
- الجدول 2-10: مقارنة تحليلية بين الأعمال السابقة والنظام المقترح 29

قائمة الاختصارات

الاختصار	المصطلح الكامل	المعنى بالعربية
API	Application Programming Interface	واجهة برمجة التطبيقات
MCP	Model Context Protocol	بروتوكول سياق النموذج
PDF	Portable Document Format	تنسيق المستندات المحمولة
HTML	HyperText Markup Language	لغة ترميز النص التشعبي
Markdown (MD)	Markdown Markup Language	لغة ماركداون
REST	Representational State Transfer	أسلوب نقل حالة التمثيل
REST API	Representational State Transfer API	واجهة برمجة تطبيقات REST
JWT	JSON Web Token	رمز ويب بصيغة JSON
JSON	JavaScript Object Notation	ترميز كائنات جافاسكربت
SVG	Scalable Vector Graphics	الرسومات المتجهة القابلة للتدرج
RTL	Right-to-Left	اتجاه الكتابة من اليمين إلى اليسار
LTR	Left-to-Right	اتجاه الكتابة من اليسار إلى اليمين
BiDi	Bidirectional Text	النصوص ثنائية الاتجاه
GUI	Graphical User Interface	واجهة المستخدم الرسومية
DOM	Document Object Model	نموذج كائن المستند
CSS	Cascading Style Sheets	أوراق الأنماط المتتالية
CDN	Content Delivery Network	شبكة توصيل المحتوى
CI/CD	Continuous Integration / Continuous Delivery	التكامل المستمر والتسليم المستمر
CLI	Command Line Interface	واجهة سطر الأوامر
SaaS	Software as a Service	البرمجيات كخدمة
IaaS	Infrastructure as a Service	البنية التحتية كخدمة
PaaS	Platform as a Service	المنصة كخدمة
FaaS	Function as a Service	الوظائف كخدمة
PoP	Point of Presence	نقطة التواجد
D1	Cloudflare D1 Database	قاعدة بيانات D1 من Cloudflare
KaTeX	KaTeX Mathematical Rendering Engine	محرك عرض المعادلات الرياضية

أداة إنشاء مخططات UML النصية	Plant Unified Modeling Language	PlantUML
لغة النمذجة الموحدة	Unified Modeling Language	UML
مكتبة تشغيل المتصفح الرئيسي	Puppeteer Headless Browser	Puppeteer
متصفح كروميوم	Chromium Browser	Chromium
البيانات الوصفية للمستند	Front Matter Metadata	Frontmatter
الحوسبة الطرفية	Edge Computing	Edge Computing
الحوسبة بدون خوادم	Serverless Computing	Serverless
منصة Cloudflare السحابية	Cloudflare Platform	Cloudflare

المخلص

في ظل التوجه المتزايد نحو كتابة الوثائق التقنية والمحتوى الرقمي باستخدام لغة التوصيف Markdown، تبرز الحاجة إلى أدوات موثوقة لتحويل هذه المستندات إلى صيغة PDF احترافية. تواجه الأنظمة التقليدية تحديات جوهرية في دعم اللغات ذات الاتجاه من اليمين لليساار (RTL) كالعربية، ودمج المخططات البيانية (PlantUML)، والمعادلات الرياضية المعقدة (LaTeX) داخل مخرجات PDF بشكل موحد. تقدم هذه الأطروحة مشروعاً متكاملًا يعمل بالكامل على البنية التحتية Serverless عبر منصة Cloudflare، يتجاوز هذه القيود ويوفر محرك تحويل عالي الأداء والقابلية للتوسع.

يعتمد النظام على واجهة برمجة تطبيقات (REST API) وواجهة ويب تفاعلية، حيث تبدأ عملية التحويل بتحليل البيانات الوصفية لمستند Markdown (Frontmatter)، ثم تحويل المحتوى إلى HTML مع تلوين النصوص البرمجية باستخدام Highlight.js، وعرض المعادلات الرياضية بواسطة KaTeX. كما يعالج النظام مخططات PlantUML من خلال جلبها من الخادم ودمجها بصيغة الرسوميات المتجهة (SVG) داخل المستند، بما يضمن الحفاظ على جودتها ووضوحها في ملف PDF النهائي. وفي المرحلة الأخيرة، يُولّد ملف PDF باستخدام تقنية Cloudflare Browser Rendering (Puppeteer) مع ضمان دعم الاتجاه من اليمين إلى اليسار (RTL) وإظهار الخلفيات المطبوعة.

الكلمات المفتاحية: تحويل المستندات، ماركداون (Markdown)، PDF، بنية Serverless، كلاودفلير (Cloudflare)، دعم اللغة العربية (RTL)، كايتركس (KaTeX)، بلانت (PlantUML) UML، واجهة برمجة التطبيقات (API).

Design and Implementation of an API and MCP Tool for Converting Markdown Documents to PDF

ABSTRACT

With the increasing trend towards writing technical documentation and digital content using Markdown, the need for reliable tools to convert these documents into professional PDF formats has become essential. Traditional systems face significant challenges in supporting Right-to-Left (RTL) languages like Arabic, embedding inline diagrams (PlantUML), and rendering complex mathematical equations (LaTeX) uniformly within PDF outputs. This thesis presents a fully integrated project operating on a Serverless infrastructure via the Cloudflare platform, overcoming these limitations and providing a high-performance, scalable conversion engine.

The system relies on a REST API and an interactive web interface. The conversion process operates through a comprehensive pipeline: starting with Frontmatter parsing, followed by converting text to HTML with syntax highlighting (Highlight.js) and mathematical equation rendering (KaTeX). The system processes PlantUML diagrams by fetching them server-side and embedding them as inline SVGs to ensure clarity in the final document. Finally, PDF generation is executed using Cloudflare Browser Rendering (Puppeteer), ensuring full RTL support and background printing.

The system features a robust security architecture, including user authentication (JWT), password hashing, and conversion history management via a D1 database. The results demonstrate that the system is capable of producing high-quality PDF documents with multiple theme options and optimal response times, making it an ideal solution for developers and writers that eliminates the need for traditional server maintenance.

Keywords: Document Conversion, Markdown, PDF, Serverless Architecture, Cloudflare, Right-to-Left (RTL), KaTeX, PlantUML, RESTful API.

الفصل الأول: المقدمة

1-1 تمهيد

شهدت السنوات الأخيرة تطوراً متسارعاً في مجال كتابة الوثائق التقنية والتوثيق البرمجي، حيث انتقل التركيز من معالجات النصوص التقليدية ذات التنسيق المعقد إلى لغات التوصيف الخفيفة (Lightweight Markup Languages). برزت لغة Markdown كبديل مثالي ومعياري صناعي معتمد، لكونها تتيح للمطورين وكتاب المحتوى فصل محتوى النص عن التنسيق البصري، مما يضمن كتابة سريعة ومركزة على البنية المنطقية للوثيقة وقابلة للتتبع عبر أنظمة التحكم بالإصدارات [1].

وعلى الرغم من الكفاءة العالية التي توفرها لغة Markdown في مرحلة التأليف والتعديل، تظل الحاجة ماسة لتحويل هذه النصوص الرقمية إلى مستندات رسمية بصيغة PDF (Portable Document Format). تتميز هذه الصيغة بقدرتها العالية على الحفاظ على التخطيط البصري والتنسيق عبر مختلف الأجهزة ومنصات التشغيل، مما يجعلها المعيار الأمثل والوسيلة الموثوقة لمشاركة الوثائق التقنية، توقيع العقود، وأرشفتها الرقمية على المدى الطويل [2].

بالتوازي مع هذا التطور في أدوات كتابة وتنسيق المحتوى، شهدت هندسة البرمجيات تحولاً جذرياً نحو تبني البنية التحتية Serverless (الحوسبة بدون خوادم تقليدية). تتيح هذه البنية للمطورين تنفيذ النص البرمجي استجابةً للأحداث دون الحاجة لإدارة أو صيانة البنية التحتية للخوادم، مما يقلل من التكاليف التشغيلية ويزيد من كفاءة التوسع التلقائي (Auto-scaling) [3]. وتبرز منصات مثل Cloudflare كرواد في هذا المجال من خلال الاعتماد على شبكات الحافة (Edge Computing)، حيث تتم معالجة الطلبات في مراكز بيانات جغرافياً قريبة من المستخدم النهائي. هذا النهج لا يوفر بيئة تنفيذ آمنة ومعزولة فحسب، بل يضمن أيضاً زمن استجابة فائق السرعة وموثوقية عالية في معالجة المهام المكثفة مثل توليد مستندات PDF ديناميكياً [3].

2-1 المشكلة البحثية وفكرة المشروع

على الرغم من الانتشار الواسع لأدوات تحويل لغة Markdown إلى صيغة PDF، إلا أن هذه الحلول تعاني من قصور جوهري يحد من كفاءتها وشموليبتها في بيئات الإنتاج المعقدة. تتمثل المشكلة الأولى في الاعتماد المفرط للعديد من هذه

الأدوات على البنية التحتية التقليدية للخوادم (Monolithic Servers)، مما يفرض تكاليف تشغيلية مرتفعة وتعقيداً في إدارة الصيانة، ويحد من قدرة النظام على التوسع المرن (Auto-scaling) لمواجهة الأحمال المتغيرة [3].

وتتجلى المشكلة الثانية والأكثر تأثيراً على تجربة المستخدم في الفجوة الكبيرة لدعم اللغات ذات الاتجاه من اليمين لليمن (RTL)، كاللغة العربية. تقبل غالبية محركات التحويل الحالية في تطبيق قواعد التنضيد ثنائي الاتجاه (Bidirectional Typesetting) بشكل صحيح، مما يؤدي إلى انعكاس ترتيب الكلمات، واختلاط النص العربي بالنصوص البرمجية، وإنتاج مستندات مشوهة غير صالحة للاستخدام المهني أو الأكاديمي [4].

إضافة إلى ذلك، تواجه أنابيب التحويل (Conversion Pipelines) الحالية صعوبة بالغة في معالجة العناصر الديناميكية والمعقدة. فعلى الرغم من أهمية المخططات البيانية (PlantUML) والمعادلات الرياضية (LaTeX) في التوثيق التقني والعلمي، تعجز معظم الأدوات عن دمجها بشكل سليم؛ حيث يتم تحويل المخططات غالباً إلى صور نقطية (Rasterized Images) منخفضة الدقة بدلاً من دمجها كرسوميات متجهة قابلة للتطوير (SVG)، كما يتم تجاهل أو تفسير خاطئ للمعادلات الرياضية أثناء توليد PDF [5].

لمعالجة هذه الفجوات البحثية والعملية، يطرح هذا المشروع فكرة بناء نظام سحابي متكامل يعمل بالكامل على بنية Serverless باستخدام منصة Cloudflare. يقوم النظام بتحويل Markdown إلى PDF عبر واجهة برمجة تطبيقات (REST API) وواجهة ويب تفاعلية. ولتجاوز القصور السابق، صُمم محرك النظام ليتضمن خطوات معالجة مسبقة (Pre-processing) تضمن دعماً كاملاً لاتجاه RTL، ودمجاً لمخططات PlantUML كرسوميات SVG مدمجة (Inline SVG) لضمان الدقة العالية، فضلاً عن تفسير المعادلات الرياضية عبر KaTeX وتلوين النصوص البرمجية باستخدام Highlight.js، ليتم بعد ذلك توليد PDF نهائياً عبر تقنية Cloudflare Browser Rendering بجودة واحترافية عاليتين وبدون أعباء تشغيل الخوادم [3].

1-3 أهداف المشروع

يهدف هذا المشروع إلى تحقيق الغايات والأهداف المحددة التالية:

1. تطوير محرك تحويل فعال من Markdown إلى PDF يعمل بالكامل على بيئة Cloudflare Serverless.

2. تصميم نظام معالجة سيرفر-سايد (Server-side) لدمج مخططات PlantUML كرسوميات متجهة (Inline SVG) داخل المستند قبل التحويل النهائي.
3. توفير دعم تنضيد كامل للغة العربية (RTL) والمعادلات الرياضية (KaTeX) وتلوين النصوص البرمجية باستخدام Highlight.js في مخرجات PDF.
4. بناء واجهة برمجة تطبيقات (REST API) آمنة وموثقة تدعم المصادقة (JWT) وإدارة سجل التحويلات.
5. نشر النظام كتطبيق ويب سريع وآمن يستفيد من شبكة توزيع المحتوى (CDN) الخاصة بـ Cloudflare.

1-4 أهمية المشروع

تتبلور أهمية هذا المشروع في بعدين أساسيين يكملان بعضهما البعض، وهما الأهمية العلمية والتقنية من جهة، والأهمية العملية من جهة أخرى:

• الأهمية العلمية والتقنية:

تتجلى الأهمية العلمية لهذا المشروع في إثبات فعالية وقابلية دمج تقنيات تصيير المتصفح (Browser Rendering) المعتمدة على Puppeteer مع البنية التحتية Serverless لتوليد ملفات ديناميكية معقدة. تاريخياً، عانت بيئات Serverless من قيود في التعامل مع العمليات المعقدة والمكثفة حسابياً (Stateful Workloads) مثل تشغيل متصفح رأسي (Headless Browser) وتوليد ملفات PDF تحتوي على عناصر رسومية متقدمة كرسوميات SVG والمعادلات الرياضية المعقدة (KaTeX) [2]. يقدم هذا المشروع دليلاً مفاهيمياً وعملياً على إمكانية تنفيذ هذه المهام الثقيلة على حافة الشبكة (Edge) دون الحاجة لإدارة خوادم تقليدية، مما يساهم بشكل مباشر في تبسيط نماذج هندسة البرمجيات، وتقليل التعقيد التشغيلي (Operational Complexity)، والانتقال نحو هياكل برمجية أكثر مرونة وقابلية للتوسع [3].

• الأهمية العملية:

على الصعيد العملي، يوفر المشروع حلاً موثقاً ومنخفض التكلفة للمجتمع العربي التقني، والذي طالما عانى من نقص الأدوات التي تدعم لغته بتنضيد سليم. فمن خلال معالجة التحديات الزمنية المتعلقة بالتنضيد ثنائي الاتجاه (BiDi) وضمان خلو المستندات من التشوهات التنسيقية، يُمكن النظام الباحثين والمطورين من إنتاج وثائق احترافية بلغتهم الأم بسهولة [4].

بالإضافة إلى ذلك، تبرز الأهمية العملية في توفير واجهات برمجة تطبيقات (REST API) مُغلقة ومؤمنة، مما يسمح لأي مطور أو مؤسسة بدمج خدمة التحويل بسلاسة ضمن تطبيقاتهم الخاصة، سواء في منتجات البرمجيات كخدمة (SaaS) أو أنظمة أتمتة الوثائق المستمرة (Continuous Documentation)، دون الحاجة للاستثمار في بنية تحتية مكلفة [5].

1-5 الفئة المستهدفة والمستفيدون

يستهدف هذا المشروع شريحة واسعة من المستفيدين الذين يبحثون عن حلول موثوقة وسريعة لتحويل وتنسيق مستنداتهم، وتشمل الفئات الرئيسية:

- **المطورون ومهندسو البرمجيات:** لتسهيل إنشاء التوثيق التقني (Documentation) لمشاريعهم البرمجية بسرعة وسلاسة عبر واجهة برمجة التطبيقات (API).
- **كُتّاب المحتوى التقني والمدونون:** لتمكينهم من إنتاج كتب إلكترونية ومقالات بصيغة PDF احترافية ومتناسقة، بالاستناد إلى مسوداتهم المكتوبة بلغة Markdown.
- **الباحثون والطلاب (خاصة في التخصصات التقنية والرياضية):** لتوفير بيئة كتابة تدعم المعادلات الرياضية (LaTeX) والمخططات البيانية (PlantUML)، مع إمكانية تصدير أوراقهم البحثية وملاحظاتهم بتنسيق علمي وأنيق.
- **الشركات وفرق تطوير المنتجات:** لدمج خاصية توليد التقارير أو العقود آلياً بصيغة PDF ضمن أنظمتهم الداخلية وسير العمل (Workflows) باستخدام واجهة برمجة التطبيقات (API) المتاحة.

الفصل الثاني: الأسس النظرية

والتقنيات المعتمدة

1-2 المفاهيم النظرية الأساسية

1-1-2 لغات التوصيف الخفيفة ومنهجية الوثائق كنصوص برمجية (Docs-as-Code)

شهدت بيئات كتابة المحتوى الرقمي وتطوير البرمجيات تحولاً جوهرياً في آليات إدارة وتوثيق المعرفة، حيث انتقل التركيز من أنظمة التنسيق الثقيلة والمعقدة إلى ما يُعرف بلغات التوصيف الخفيفة (Lightweight Markup Languages). تُعرف لغة التوصيف الخفيفة بأنها لغة ترميز نصية بسيطة تستخدم بناءً نحويًا غير معقد لإضافة علامات التنسيق (Formatting Tags) إلى النص العادي، مع الحفاظ على قابلية قراءة النص وفهمه من قبل الإنسان حتى قبل معالجته أو عرضه (Rendering) [1].

تُعد لغة Markdown، التي ابتكرها جون غروبر (John Gruber) عام 2004، التجسيد الأبرز لهذا المفهوم، إذ تعتمد فلسفتها على مبدأ فصل المحتوى عن التنسيق (Separation of Content and Presentation)، بحيث يركز الكاتب على البنية المنطقية للمستند، مثل العناوين والفقرات والقوائم، باستخدام رموز بسيطة وسهلة الفهم، بدلاً من الاهتمام بالتنسيق المرئي المباشر كما هو الحال في معالجات النصوص أو التعامل مع البنية المعقدة للغات مثل HTML و LaTeX. ويسهم هذا الأسلوب في تبسيط عملية كتابة الوثائق وتحسين قابليتها للصيانة وإعادة الاستخدام [1].

يوضح الجدول (1-2) مقارنة بين أكثر أساليب كتابة المستندات استخداماً، من حيث فلسفة التصميم وسهولة التعلم وقابلية قراءة النصوص المصدرية.

الجدول 1-2: مقارنة بين أنظمة كتابة وتنسيق المستندات

نظام التنسيق	فلسفة التصميم	قابلية القراءة البشرية للنص المصدري	منحنى التعلم	الاستخدامات الشائعة
معالجات النصوص (WYSIWYG)	دمج المحتوى مع التنسيق المرئي	منخفضة	منخفض	المستندات المكتبية والتقارير العامة
HTML / LaTeX	فصل المحتوى عن التنسيق باستخدام بناء نحوي متقدم	متوسطة إلى منخفضة	مرتفع	صفحات الويب والنشر الأكاديمي

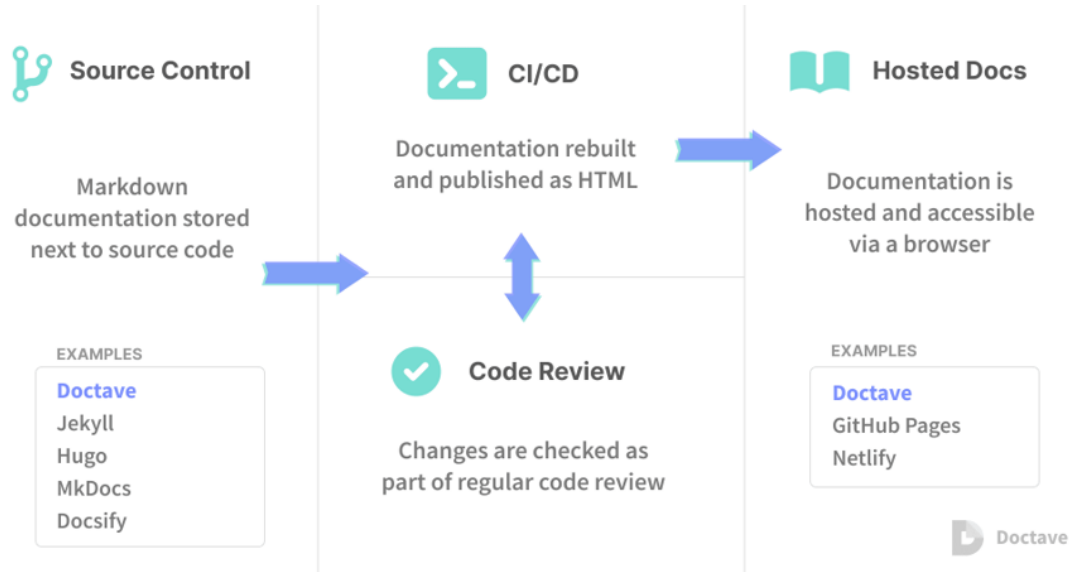
التوثيق البرمجي، ملفات README، المواقع الثابتة	منخفض جداً	مرتفعة	فصل المحتوى عن التسويق باستخدام صياغة بسيطة	Markdown
--	------------	--------	---	----------

يتضح من الجدول السابق أن لغة Markdown تمتاز بسهولة تعلمها ووضوح نصوصها المصدرية مقارنةً بالبدايل الأخرى، الأمر الذي جعلها الخيار الأكثر انتشاراً في كتابة الوثائق التقنية الحديثة.

أدى الانتشار الواسع لـ Markdown إلى ظهور منهجية هندسية حديثة تُعرف باسم الوثائق كنصوص برمجية (Docs-as-Code)، والتي تقوم على التعامل مع الوثائق التقنية بالطريقة نفسها التي يتم بها التعامل مع النصوص البرمجية المصدرية. ووفقاً لهذه المنهجية تُكتب الوثائق باستخدام Markdown، وتُحفظ داخل مستودعات التحكم بالإصدارات مثل Git، ثم تخضع لعمليات المراجعة (Code Review)، وإدارة الإصدارات، وعمليات الدمج (Merge)، كما يمكن نشرها بصورة آلية من خلال خطوط التكامل والتسليم المستمر [1] (CI/CD Pipelines).

وقد ساهم هذا الأسلوب في إزالة الفجوة التقليدية بين فرق التطوير وفرق التوثيق، إذ أصبح بإمكان المطورين تحديث الوثائق بالتزامن مع تطوير النص البرمجي داخل بيئة العمل نفسها، مما يحافظ على توافق الوثائق مع النظام البرمجي ويزيد من جودة التوثيق واستمرارية تحديثه، وهو ما جعل منهجية Docs-as-Code معياراً معتمداً في مشاريع البرمجيات الحديثة [1].

يوضح الشكل (1-2) دورة العمل الأساسية في منهجية Docs-as-Code، بدءاً من كتابة ملفات Markdown داخل مستودع النصوص البرمجية المصدرية، ثم مراجعتها وإعادة بنائها عبر خطوط CI/CD، وانتهاءً بنشر الوثائق بصيغة HTML لتصبح متاحة للمستخدمين.



الشكل 2-1: سير العمل في منهجية الوثائق كنصوص برمجية (Docs-as-Code Workflow)

يتضح من الشكل (2-1) أن دورة حياة الوثيقة أصبحت جزءاً لا يتجزأ من دورة حياة البرمجيات؛ إذ تبدأ بكتابة ملفات Markdown داخل مستودع المشروع، ثم تمر بمرحلة مراجعة التعديلات ضمن عمليات مراجعة النصوص البرمجية (Code Review)، وبعد اعتمادها تُعاد عملية البناء تلقائياً بواسطة خطوط التكامل والتسليم المستمر (CI/CD)، ليتم في النهاية نشر الوثائق بصيغة HTML وإتاحتها للمستخدمين عبر المتصفح. ويؤدي هذا الأسلوب إلى تقليل الأخطاء، وضمان تزامن الوثائق مع الإصدارات البرمجية المختلفة، بالإضافة إلى تسريع عمليات النشر والتحديث المستمر.

2-1-2 الحوسبة بدون خوادم والحوسبة الطرفية (Serverless & Edge Computing)

شهدت هندسة البرمجيات خلال العقد الأخير تحولاً جذرياً في أساليب توفير الموارد الحاسوبية وإدارة التطبيقات السحابية، حيث انتقلت من الاعتماد على الخوادم التقليدية إلى نماذج أكثر مرونة تعتمد على الخدمات السحابية الحديثة. ويُعد مفهوم الحوسبة بدون خوادم (Serverless Computing) أحد أبرز هذه النماذج، إذ يتيح للمطورين التركيز على تطوير التطبيقات دون الحاجة إلى إدارة البنية التحتية أو صيانة الخوادم، بينما يتولى مزود الخدمة السحابية جميع العمليات المتعلقة بإدارة الموارد، والتوسع، والتحديث، والمراقبة [3].

وعلى الرغم من أن مصطلح "بدون خوادم" قد يوحي بعدم وجود خوادم فعلية، إلا أن الواقع يتمثل في وجود خوادم تُدار بالكامل بواسطة مزود الخدمة السحابية، بينما لا يتعامل المطور إلا مع النص البرمجي التي تُنفذ استجابةً لحدث معين (Event) أو طلب من المستخدم (Trigger). ويؤدي هذا النموذج إلى تقليل الأعباء التشغيلية (Operational Overhead)، كما يتيح الدفع مقابل زمن التنفيذ الفعلي فقط، بدلاً من تشغيل الخوادم بصورة دائمة، الأمر الذي يساهم في خفض التكاليف وتحسين كفاءة استهلاك الموارد [3].

ولتوضيح الاختلاف بين النماذج المختلفة للحوسبة السحابية، يبين الجدول (2-2) توزيع المسؤوليات بين المطور ومزود الخدمة في كل من البنية التحتية كخدمة (IaaS)، والمنصة كخدمة (PaaS)، والحوسبة بدون خوادم (Serverless).

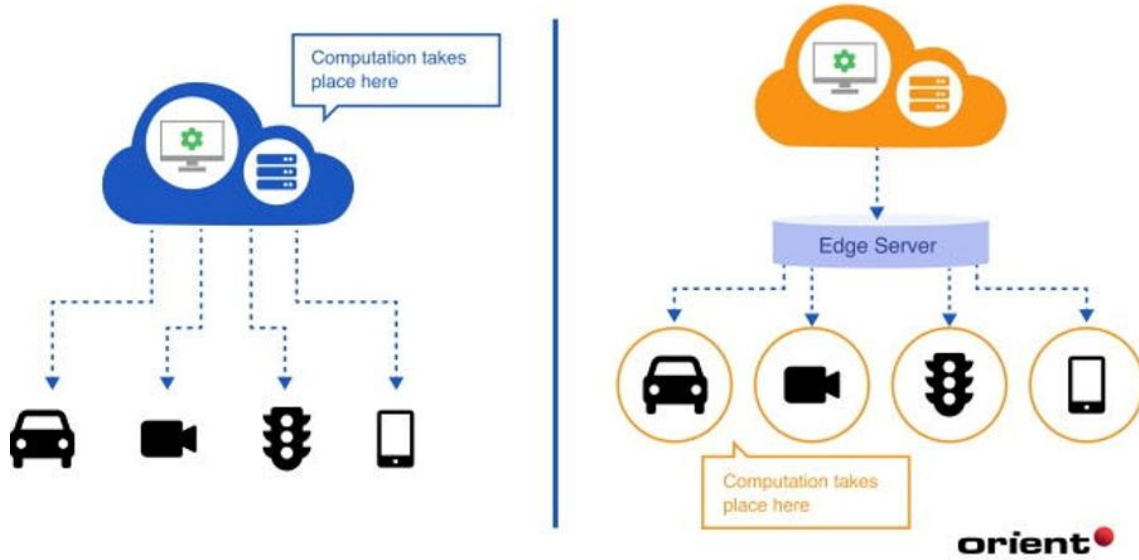
الجدول 2-2: مقارنة مسؤوليات المطور بين نماذج الحوسبة السحابية المختلفة

مستوى المسؤولية	IaaS	PaaS	Serverless (FaaS)
إدارة الشبكات والبنية الفيزيائية	مزود الخدمة	مزود الخدمة	مزود الخدمة
إدارة نظام التشغيل	المطور	مزود الخدمة	مزود الخدمة
إدارة الخوادم	المطور	جزئياً	مزود الخدمة
التوسع (Scaling)	يدوي أو برمجي	تلقائي جزئياً	تلقائي بالكامل
إدارة السعة (Capacity Planning)	المطور	المطور	مزود الخدمة
نشر التطبيق	المطور	المطور	المطور

يتبين من الجدول (2-2) أن نموذج Serverless يقلل بصورة كبيرة من مسؤوليات المطور مقارنة بالنماذج التقليدية، حيث يقتصر دوره على تطوير التطبيق ونشره، بينما تتولى المنصة جميع العمليات المتعلقة بإدارة البنية التحتية والتوسع والصيانة. وبالتزامن مع انتشار Serverless ظهر مفهوم الحوسبة الطرفية (Edge Computing)، والذي يهدف إلى تقليل زمن الاستجابة (Latency) من خلال تنفيذ العمليات الحاسوبية بالقرب من المستخدم النهائي بدلاً من إرسال جميع الطلبات إلى مراكز بيانات مركزية بعيدة [3]. ويعتمد هذا النموذج على نشر العقد الحاسوبية (Edge Nodes) في مواقع جغرافية متعددة، بحيث تتم معالجة الطلبات داخل أقرب نقطة وجود (Point of Presence - PoP)، مما يقلل من زمن انتقال البيانات ويحسن أداء التطبيقات ذات الزمن الحقيقي.

ويعد الدمج بين الحوسبة بدون خوادم والحوسبة الطرفية أحد أحدث الاتجاهات في هندسة البرمجيات السحابية، حيث تُنفذ الوظائف البرمجية مباشرة على حافة الشبكة (Edge)، وهو ما يوفر استجابة أسرع، ويزيد من موثوقية الخدمة، ويخفف الضغط على مراكز البيانات المركزية، إضافة إلى تحسين تجربة المستخدم النهائي، خاصة في التطبيقات التي تتطلب معالجة فورية مثل خدمات الذكاء الاصطناعي، وتوليد المستندات، وإنترنت الأشياء [3].

Cloud Computing vs Edge Computing



الشكل 2-2: مقارنة بين الحوسبة السحابية (Cloud Computing) والحوسبة الطرفية (Edge Computing)

يتضح من الشكل (2-2) أن الحوسبة السحابية التقليدية تعتمد على إرسال جميع الطلبات إلى مركز بيانات مركزي، مما يزيد من زمن الاستجابة مع ازدياد المسافة بين المستخدم والخادم. أما في الحوسبة الطرفية، فتتم معالجة البيانات داخل أقرب عقدة طرفية، وهو ما يقلل زمن الانتقال، ويحسن أداء التطبيقات، ويزيد من سرعة الاستجابة، وهو النهج الذي تعتمد عليه منصة Cloudflare المستخدمة في هذا المشروع لتوفير خدمة تحويل ملفات Markdown إلى PDF بكفاءة عالية.

3-1-2 عرض المتصفحات الرأسية (Headless Browser Rendering)

مع التطور المتسارع لتطبيقات الويب الحديثة، أصبحت معظم صفحات الويب تعتمد على محتوى ديناميكي يُنشأ من خلال تنفيذ تعليمات JavaScript وتطبيق أنماط CSS بصورة آنية، مما جعل عملية تحويل صفحات الويب إلى مستندات ثابتة

أكثر تعقيداً من الأساليب التقليدية. وللتغلب على هذه التحديات ظهر مفهوم المتصفح الرأسي (Headless Browser)، وهو متصفح ويب يعمل بجميع وظائفه الأساسية دون توفير واجهة مستخدم رسومية (Graphical User Interface - GUI)، حيث يتم التحكم به بالكامل برمجياً عبر واجهات برمجة التطبيقات، الأمر الذي يجعله مناسباً لتطبيقات الأتمتة واختبار البرمجيات وتوليد المستندات [2].

يعتمد المتصفح الرأسي على محرك العرض (Rendering Engine) نفسه المستخدم في المتصفحات التقليدية، حيث يقوم بتحميل ملفات HTML، وتطبيق أنماط CSS، وتنفيذ تعليمات JavaScript، ثم بناء نموذج كائن المستند (Document Object Model - DOM) وشجرة العرض (Render Tree). إلا أن الفرق الجوهرى يتمثل في أن النتائج النهائية لا تُعرض على شاشة المستخدم، وإنما تُستخدم برمجياً لإنشاء مخرجات أخرى مثل لقطات الشاشة أو ملفات PDF، مع الحفاظ على التوافق الكامل مع معايير الويب الحديثة [2].

ولتوضيح الفروقات الأساسية بين المتصفحات التقليدية والمتصفحات الرأسية، يبين الجدول (2-3) مقارنة بين خصائص كل منهما.

الجدول 2-3: مقارنة بين المتصفحات التقليدية والمتصفحات الرأسية

خاصية المقارنة	المتصفح التقليدي (GUI)	المتصفح الرأسي (Browser Headless)
واجهة المستخدم	متوفرة	غير متوفرة
أسلوب التشغيل	يعتمد على تفاعل المستخدم	يعتمد على الأوامر البرمجية
تنفيذ العمليات	يدوي	آلي بالكامل
استهلاك الموارد	مرتفع نسبياً	أقل نسبياً
بيئة التشغيل	أجهزة المستخدم النهائية	الخوادم وبيئات CI/CD
أبرز الاستخدامات	تصفح مواقع الويب	الاختبارات الآلية، استخراج البيانات، توليد ملفات PDF

يتضح من الجدول (2-3) أن المتصفح الرأسي لا يُعد بديلاً للمتصفح التقليدي، وإنما يمثل بيئة تنفيذ آلية موجهة للتطبيقات والخوادم، حيث يمكنه تنفيذ جميع عمليات عرض صفحات الويب دون الحاجة إلى وجود مستخدم أو واجهة رسومية.

وتبرز أهمية المتصفحات الرأسية بصورة خاصة في أنظمة توليد المستندات الرقمية، إذ تعتمد العديد من أدوات التحويل التقليدية على تحليل ملفات HTML فقط دون تنفيذ تعليمات JavaScript أو دعم كامل لتقنيات CSS3 الحديثة، مما يؤدي إلى اختلاف واضح بين الصفحة الأصلية وملف PDF الناتج. أما المتصفح الرأسي فيقوم بمحاكاة عملية عرض الصفحة كما لو كانت مفتوحة داخل متصفح حقيقي، ثم يحول الحالة النهائية للمحتوى إلى مستند PDF مطابق لما يراه المستخدم داخل المتصفح، وهو ما يضمن المحافظة على التخطيط والتنسيق وجميع العناصر الديناميكية [2].

ويُعرف هذا الأسلوب باسم العرض من جانب الخادم (Server-Side Rendering)، حيث يبدأ النظام بتحميل الصفحة الافتراضية، ثم تنفيذ جميع الملفات البرمجية المرتبطة بها، بما في ذلك عرض المعادلات الرياضية، وتطبيق تلوين النصوص البرمجية، وتحميل المخططات البيانية المضمنة، وبعد التأكد من اكتمال تحميل الصفحة واستقرارها، يتم استدعاء واجهة الطباعة الخاصة بالمتصفح لإنتاج ملف PDF نهائي عالي الجودة. ويؤدي هذا الأسلوب إلى تحقيق ما يُعرف بالدقة المطابقة للعرض (Pixel-perfect Accuracy)، بحيث يكون المستند الناتج مطابقاً تقريباً لما يظهر داخل المتصفح التقليدي [2].

ويُعد هذا النهج من أهم التقنيات المستخدمة في المشروع الحالي، حيث يعتمد النظام على خدمة Cloudflare Browser Rendering لتشغيل متصفح Chromium الرأسي داخل بيئة Serverless، الأمر الذي يتيح إنشاء ملفات PDF احترافية تدعم عناصر الويب الحديثة، مع المحافظة على جودة التخطيط، وسرعة التنفيذ، وانخفاض التكلفة التشغيلية.

2-1-4 تخطيط النصوص ثنائية الاتجاه (Bidirectional Text - BiDi)

أصبحت تطبيقات الويب الحديثة وأنظمة توليد المستندات تدعم عدداً كبيراً من اللغات العالمية، مما جعل التعامل مع النصوص متعددة اللغات ضرورة أساسية في تصميم البرمجيات. وتبرز إحدى أهم التحديات عند احتواء المستند الواحد على لغات تختلف في اتجاه الكتابة، مثل اللغة العربية التي تُكتب من اليمين إلى اليسار (Right-to-Left - RTL)، واللغة الإنجليزية التي تُكتب من اليسار إلى اليمين (Left-to-Right - LTR). ويُعرف هذا النوع من المحتوى باسم النصوص ثنائية الاتجاه (Bidirectional Text - BiDi) [4].

لا يكفي تحديد اتجاه الصفحة بالكامل لمعالجة هذا النوع من النصوص، لأن السطر الواحد قد يحتوي على كلمات عربية، وأرقام، ومصطلحات إنجليزية، أو حتى نصوص برمجية برمجية، ولكل منها اتجاه مختلف. فعلى سبيل المثال، عند كتابة

جملة عربية تتضمن اسم دالة برمجية أو رابطاً إلكترونياً، يجب أن تُعرض الكلمات العربية باتجاه اليمين إلى اليسار، بينما يبقى النص الإنجليزي محتفظاً باتجاهه الطبيعي من اليسار إلى اليمين، دون أن يؤثر أحدهما في الآخر [4].

ولتحقيق ذلك، يعتمد نظام Unicode على خوارزمية قياسية تُعرف باسم Unicode Bidirectional Algorithm، والتي تقوم بتحليل كل محرف داخل النص وتحديد اتجاه عرضه وفقاً لسياق الفقرة وأنواع المحارف المحيطة به، مما يسمح بعرض النصوص المختلطة بصورة صحيحة في معظم الحالات [4].

ولتوضيح أنواع المحارف المستخدمة في هذه الخوارزمية، يعرض الجدول (2-4) تصنيف المحارف وفقاً لتأثيرها في اتجاه النص.

الجدول 2-4: تصنيف المحارف في خوارزمية Unicode للنصوص ثنائية الاتجاه

نوع المحرف	الاتجاه الأساسي	تأثيره في اتجاه النص	أمثلة
المحارف القوية (Strong Characters)	RTL أو LTR	تحدد اتجاه النص وتؤثر في المحارف المجاورة	الأحرف العربية، الأحرف اللاتينية
المحارف الضعيفة (Weak Characters)	غالباً LTR	لا تحدد اتجاه النص بصورة مستقلة	الأرقام، الرموز الرقمية
المحارف المحايدة (Neutral Characters)	لا تمتلك اتجاهًا	تستمد اتجاهها من المحارف المحيطة	المسافات، الأقواس، علامات الترقيم

يتضح من الجدول (2-4) أن معالجة النصوص المختلطة لا تعتمد على اتجاه الصفحة فقط، وإنما على نوع كل محرف داخل النص، وهو ما يفسر ظهور بعض الأخطاء عند دمج العربية مع الإنجليزية أو مع النصوص البرمجية داخل المستندات التقنية.

وعلى الرغم من كفاءة خوارزمية Unicode، إلا أنها قد تواجه بعض الحالات المعقدة، مثل وجود أقواس وعلامات ترقيم بين نصوص مختلفة الاتجاه، أو إدراج مقاطع برمجية داخل فقرة عربية، مما يؤدي إلى ظهور الأقواس أو علامات الترقيم في مواقع غير صحيحة، أو انعكاس ترتيب أجزاء من النص، وهو ما يؤثر في جودة المستند النهائي ويقلل من قابليته للقراءة [4].

ولمعالجة هذه المشكلات، تستخدم أنظمة العرض الحديثة مجموعة من وسائل التحكم في الاتجاه، مثل محارف التحكم الخاصة بـ (Unicode (Directional Formatting Characters، إضافة إلى آليات العزل الاتجاهي (Directional Isolates)، والتي تسمح بعرض كل جزء من النص وفق اتجاهه الطبيعي دون التأثير بالنصوص المجاورة. وتعد هذه الآليات ضرورية عند تصميم أنظمة توليد مستندات PDF التي تدعم اللغة العربية بصورة صحيحة.

ويولي المشروع الحالي اهتماماً خاصاً بهذه النقطة، إذ يتضمن مرحلة معالجة مسبقة (Pre-processing) للنصوص قبل تحويلها إلى HTML، بهدف ضمان تطبيق قواعد BiDi بصورة صحيحة داخل ملفات PDF، والحفاظ على سلامة ترتيب الكلمات، والأرقام، والنصوص البرمجية، والرموز الخاصة.



الشكل 2-3: مقارنة بين اتجاهي الكتابة من اليسار إلى اليمين (LTR) ومن اليمين إلى اليسار (RTL)

يتضح من الشكل (2-3) أن اللغة الإنجليزية تتبع اتجاه الكتابة من اليسار إلى اليمين (LTR)، في حين تتبع اللغة العربية الاتجاه المعاكس (RTL). وعند دمج الاتجاهين داخل المستند نفسه، تظهر الحاجة إلى تطبيق خوارزميات خاصة مثل Unicode Bidirectional Algorithm لضمان عرض النصوص المختلطة بصورة صحيحة، وهو ما يمثل أحد أهم التحديات التي يعالجها النظام المقترح.

2-1-5 الرسوميات المتجهة مقابل الصور النقطية في المستندات (Vector vs. Raster Graphics)

تلعب الرسوميات الرقمية دوراً محورياً في جودة المستندات التقنية، إذ تعتمد عليها المخططات البيانية، والرسومات الهندسية، والشعارات، والرسوم التوضيحية المستخدمة في توثيق الأنظمة البرمجية. ويمكن تصنيف الرسوميات الرقمية إلى نوعين

رئيسيين هما الصور النقطية (Raster Graphics) والرسومات المتجهة (Vector Graphics)، ويختلف كل منهما في آلية التمثيل الرقمي، وجودة العرض، ومدى ملاءمته لتوليد مستندات PDF [5].

تعتمد الصور النقطية، مثل صيغ JPEG و PNG و GIF، على شبكة من وحدات البكسل (Pixels)، بحيث يخزن كل بكسل معلومات اللون والموقع داخل الصورة. وترتبط جودة الصورة النقطية ارتباطاً مباشراً بدقتها الأصلية، لذلك يؤدي تكبيرها إلى ظهور التشوهات البصرية أو ما يُعرف بظاهرة التبكسل (Pixelation)، حيث تصبح الحواف غير واضحة وتفقد الصورة جزءاً من تفاصيلها [5].

في المقابل، تعتمد الرسومات المتجهة، مثل صيغة SVG (Scalable Vector Graphics)، على معادلات رياضية تصف الأشكال والخطوط والمنحنيات بدلاً من الاعتماد على البكسلات. ويمنح هذا الأسلوب الرسومات المتجهة خاصية الاستقلالية عن الدقة (Resolution Independence)، حيث يمكن تكبيرها أو تصغيرها إلى أي حجم دون فقدان الجودة أو وضوح الحواف، مما يجعلها مناسبة للرسومات الهندسية والمخططات التقنية [5].

ويبين الجدول (2-5) مقارنة بين الرسومات النقطية والرسومات المتجهة من حيث البنية والخصائص والاستخدامات.

الجدول 2-5: مقارنة بين الرسومات النقطية والرسومات المتجهة

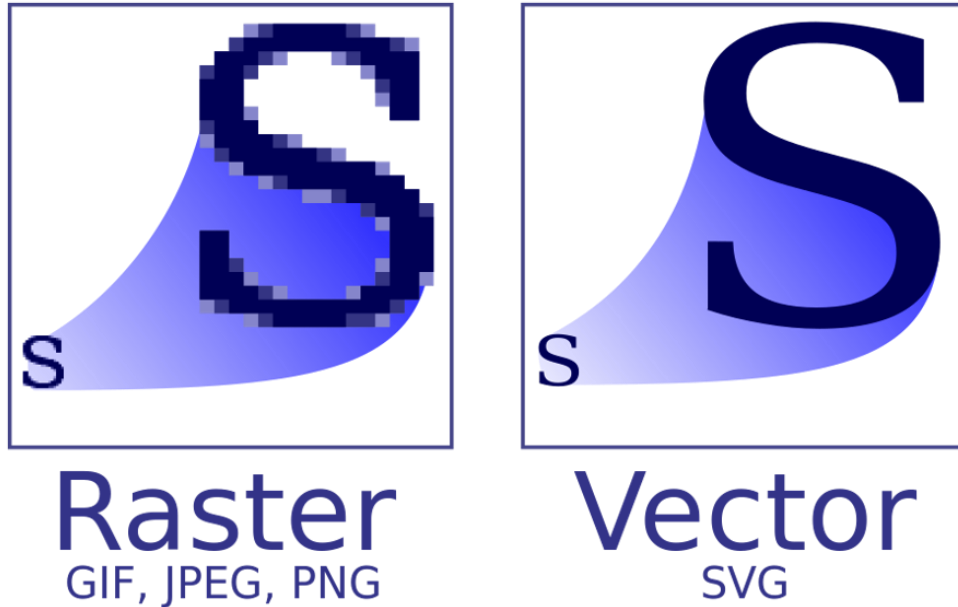
الرسومات المتجهة (Vector)	الرسومات النقطية (Raster)	خاصية المقارنة
معادلات رياضية ومسارات هندسية	شبكة من وحدات البكسل (Pixels)	البنية الأساسية
تبقى ثابتة دون فقدان الجودة	تتخفف وتظهر ظاهرة التبكسل	الجودة عند التكبير
صغير نسبياً للمخططات والرسومات	يعتمد على الأبعاد والدقة	حجم الملف
ممتازة بجميع درجات التكبير	جيدة عند الدقة العالية فقط	ملاءمة الطباعة
يمكن البحث عن النصوص المضمنة	غير مدعومة	قابلية البحث داخل PDF
الشعارات، المخططات، الرسوم الهندسية	الصور الفوتوغرافية واللقطات	الاستخدامات الشائعة

يتضح من الجدول (2-5) أن الرسومات المتجهة تتفوق بصورة واضحة في التطبيقات التي تتطلب المحافظة على جودة الرسم أثناء التكبير أو الطباعة، وهو ما يجعلها الخيار الأمثل للمخططات الفنية والبرمجية المستخدمة في المستندات التقنية.

ونكتسب هذه المقارنة أهمية كبيرة في أنظمة تحويل Markdown إلى PDF، إذ تعتمد معظم أدوات التحويل التقليدية على تحويل المخططات إلى صور نقطية قبل إدراجها داخل المستند، مما يؤدي إلى انخفاض الجودة عند الطباعة أو التكبير. أما المشروع الحالي فيعتمد على دمج مخططات PlantUML بصيغة SVG مباشرة داخل صفحة HTML قبل عملية التحويل، الأمر الذي يحافظ على جودة الرسومات ويقلل من حجم الملف النهائي [5].

وتوفر الرسومات المتجهة ميزة إضافية تتمثل في إمكانية التعامل مع النصوص المضمنة داخل الرسم كعناصر نصية حقيقية، وليس كصورة ثابتة، مما يسمح بإجراء عمليات البحث والنسخ داخل ملف PDF، بالإضافة إلى تحسين إمكانية الوصول (Accessibility) والتوافق مع برامج قراءة المستندات، وهو ما يمثل ميزة مهمة في التطبيقات الأكاديمية والتقنية [5].

واعتماداً على هذه المزايا، يستخدم المشروع الحالي الرسومات المتجهة بصيغة SVG لتمثيل مخططات PlantUML داخل المستندات، مما يضمن إنتاج ملفات PDF ذات جودة عالية، مع المحافظة على وضوح الرسومات عند التكبير والطباعة، وتقليل حجم الملفات مقارنة بالصور النقطية.



الشكل 2-4: مقارنة بين الرسومات النقطية (Raster Graphics) والرسومات المتجهة (Vector Graphics)

يتضح من الشكل (2-4) أن الرسوميات النقطية تعتمد على شبكة من البكسلات، لذلك تظهر الحواف متعرجة عند تكبير الصورة، في حين تعتمد الرسوميات المتجهة على أوصاف رياضية تسمح بالحفاظ على جودة الرسم مهما تغيرت أبعاده. ولهذا السبب تُعد صيغة SVG الخيار الأنسب لتمثيل المخططات البيانية داخل المستندات التقنية، وهو النهج الذي يتبناه النظام المقترح عند دمج مخططات PlantUML قبل تحويلها إلى ملفات PDF.

2-2 التقنيات، الأدوات، وإطارات العمل المستخدمة

يتطلب تطوير نظام يعمل بالكامل ضمن بيئة الحوسبة بدون خوادم (Serverless Computing) اختيار مجموعة متكاملة من التقنيات والأدوات البرمجية القادرة على تحقيق الأداء العالي، وقابلية التوسع، وسهولة الصيانة، مع المحافظة على انخفاض استهلاك الموارد. ونظراً لأن المشروع يعتمد على بنية Cloudflare Edge، فقد تم اختيار المكس التقني (Tech Stack) بعناية لضمان توافق جميع مكوناته مع بيئة التنفيذ الطرفية، وقدرته على معالجة مستندات Markdown وتحويلها إلى ملفات PDF بكفاءة وموثوقية. ويتناول هذا القسم البنية التحتية السحابية، وإطار العمل المستخدم في تطوير الواجهة الخلفية، ومحرك معالجة المحتوى، بالإضافة إلى تقنيات الواجهة الأمامية التي تم اعتمادها في المشروع.

1-2-2 البنية التحتية السحابية (Cloud Infrastructure)

يعتمد المشروع على منظومة Cloudflare السحابية لتوفير بيئة تشغيل متكاملة تعمل على حافة الشبكة (Edge)، حيث تجمع بين خدمات الاستضافة، وتنفيذ الوظائف السحابية، وإدارة قواعد البيانات، وتشغيل المتصفحات الرأسية ضمن منصة واحدة. ويسهم هذا التكامل في تقليل التعقيد التشغيلي الناتج عن توزيع مكونات النظام على مزودي خدمات مختلفين، كما يضمن سرعة الاستجابة وارتفاع موثوقية النظام بفضل انتشار مراكز بيانات Cloudflare حول العالم [6].

تعتمد هذه البنية على تنفيذ النصوص البرمجية داخل العقد الطرفية (Edge Nodes)، بحيث تتم معالجة طلبات المستخدم في أقرب مركز بيانات جغرافياً، مما يقلل زمن انتقال البيانات (Latency) ويحسن أداء التطبيق مقارنة بالحلول السحابية

التقليدية التي تعتمد على مراكز بيانات مركزية بعيدة. كما تتيح هذه البيئة توسعاً تلقائياً (Auto-scaling) يتكيف مع عدد الطلبات دون الحاجة إلى تدخل المطور أو إدارة الخوادم يدوياً [6].

ويتكون النظام من عدة خدمات رئيسية ضمن منظومة Cloudflare، يؤدي كل منها وظيفة محددة في عملية تحويل المستندات وإدارة البيانات، كما هو موضح في الجدول (2-6).

الجدول 2-6: مكونات البنية التحتية السحابية المستخدمة في المشروع

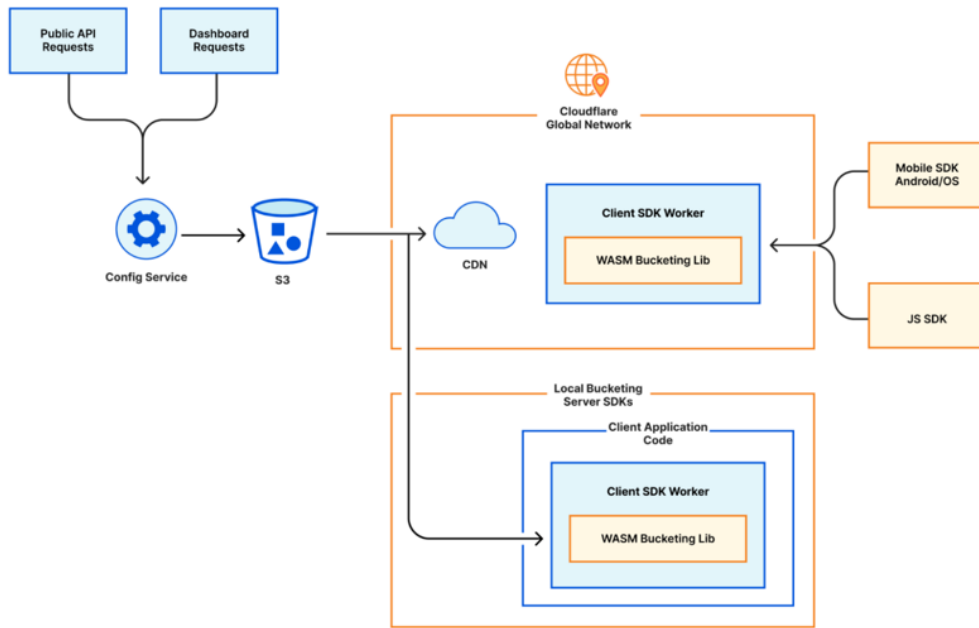
الخدمة السحابية	فئة الخدمة	الدور في المشروع	مميزات الاختيار
Cloudflare Pages	Hosting & Edge Functions	استضافة الواجهة الأمامية وتشغيل واجهات API	تكامل مباشر مع Git، سرعة نشر عالية، دعم Edge Functions
Cloudflare Browser Rendering	Headless Browser as a Service	تشغيل متصفح Chromium الرئيسي وتوليد ملفات PDF	عدم الحاجة لإدارة متصفح Chromium محلياً، سرعة التنفيذ
Cloudflare D1	Serverless Relational Database	تخزين بيانات المستخدمين وسجل عمليات التحويل	قاعدة بيانات موزعة جغرافياً بزمان استجابة منخفض

يتضح من الجدول (2-6) أن خدمات Cloudflare لا تعمل بصورة منفصلة، وإنما تشكل منظومة متكاملة تتبادل البيانات فيما بينها، بحيث تتولى Cloudflare Pages استقبال طلبات المستخدم، بينما تنفذ Browser Rendering عملية إنشاء ملفات PDF، وتقوم قاعدة البيانات D1 بحفظ بيانات المستخدمين وسجل عمليات التحويل، مما يوفر بيئة تشغيل متجانسة ذات أداء مرتفع.

وتعمل Cloudflare Pages بوصفها نقطة الدخول الرئيسية للتطبيق، حيث تستضيف ملفات الواجهة الأمامية وتوفر في الوقت نفسه بيئة تشغيل للوظائف البرمجية (Edge Functions)، الأمر الذي يسمح بتنفيذ منطق التطبيق دون الحاجة إلى خوادم تقليدية. كما تعتمد المنصة على التكامل المباشر مع مستودعات Git، مما يتيح نشر التحديثات بصورة تلقائية مع كل عملية تطوير جديدة [6].

أما خدمة Cloudflare Browser Rendering فتُعدّ العنصر الأساسي المسؤول عن إنشاء ملفات PDF، إذ توفر بيئة جاهزة لتشغيل متصفح Chromium الرأسي باستخدام Puppeteer، بحيث يتم تحميل صفحة HTML الناتجة عن محرك التحويل، ثم تنفيذ جميع عناصرها الديناميكية قبل إنشاء ملف PDF النهائي. ويساعد ذلك في إنتاج مستندات عالية الجودة دون الحاجة إلى تثبيت أو إدارة متصفح Chromium داخل الخادم [7].

ولإدارة البيانات، يعتمد المشروع على قاعدة البيانات Cloudflare D1، وهي قاعدة بيانات علائقية مبنية على محرك SQLite وتعمل ضمن بيئة Serverless. وتمتاز D1 بإمكانية توزيع البيانات عبر مراكز بيانات Cloudflare، مما يضمن سرعة تنفيذ عمليات المصادقة، وتخزين المستخدمين، وسجل عمليات التحويل، مع المحافظة على انخفاض زمن الاستجابة مقارنة بقواعد البيانات المركزية التقليدية [8].



الشكل 2-5: معمارية البنية التحتية السحابية المعتمدة على Cloudflare

يتضح من الشكل (2-6) أن جميع طلبات المستخدم تمر أولاً عبر شبكة Cloudflare Global Network، حيث تُوجه إلى أقرب عقدة طرفية لمعالجتها. وتعمل خدمات Cloudflare المختلفة بصورة متكاملة، إذ تستقبل الواجهة الأمامية طلبات المستخدم، ثم تُنفذ الوظائف البرمجية داخل بيئة Edge، بينما يتم الوصول إلى البيانات المخزنة وإجراء عمليات المعالجة

قبل إعادة النتيجة إلى المستخدم بزمان استجابة منخفض. ويبرز الشكل كذلك دور شبكة توزيع المحتوى (CDN) في تحسين الأداء وتسريع وصول البيانات، وهو ما ينسجم مع البنية المعتمدة في المشروع لتوفير خدمة تحويل مستندات عالية الكفاءة.

2-2-2 الخلفية وإطار عمل الويب (Backend & Web Framework)

تعتمد أنظمة الحوسبة الطرفية (Edge Computing) على بيئات تشغيل تختلف عن بيئات الخوادم التقليدية، إذ تفرض قيوداً تتعلق بحجم الحزمة البرمجية (Bundle Size)، و زمن بدء التشغيل (Cold Start)، واستهلاك الذاكرة، بالإضافة إلى محدودية بعض واجهات برمجة التطبيقات الخاصة ببيئة Node.js. لذلك، يتطلب تطوير التطبيقات الموجهة للعمل على الحافة اختيار إطار عمل خفيف يعتمد على معايير الويب الحديثة ويحقق توافقاً كاملاً مع بيئات Cloudflare Workers [9].

واستناداً إلى هذه المتطلبات، اعتمد المشروع إطار العمل Hono لتطوير الواجهة الخلفية وواجهات برمجة التطبيقات (REST API). ويتميز هذا الإطار بخفة حجمه، واعتماده على واجهات Request و Response القياسية في الويب بدلاً من الاعتماد على واجهات Node.js التقليدية، مما يجعله مناسباً للتطبيقات التي تعمل ضمن بيئات الحوسبة الطرفية. كما يوفر Hono نظام توجيه (Routing) سريعاً يدعم إنشاء واجهات برمجية عالية الأداء مع المحافظة على انخفاض زمن الاستجابة [9].

ولضمان سلامة البيانات المتبادلة بين العميل والخادم، يستخدم المشروع مكتبة Zod للتحقق من صحة المدخلات (Schema Validation)، حيث تقوم بفحص جميع البيانات الواردة إلى واجهات برمجة التطبيقات قبل تمريرها إلى منطق التطبيق. ويساعد ذلك في اكتشاف الأخطاء مبكراً ومنع إدخال بيانات غير صحيحة أو غير مكتملة، مما يزيد من موثوقية النظام ويحسن من جودة معالجة الطلبات [10].

أما فيما يتعلق بالمصادقة وإدارة المستخدمين، فقد تم الاعتماد على تقنية JSON Web Tokens (JWT)، والتي تعد من أكثر أساليب المصادقة استخداماً في التطبيقات الحديثة. وتعتمد هذه التقنية على إنشاء رمز رقمي يحتوي على بيانات المستخدم، ويُرسل مع كل طلب للتحقق من الهوية دون الحاجة إلى إنشاء جلسات (Sessions) داخل الخادم، الأمر الذي يجعلها مناسبة بصورة خاصة لتطبيقات Serverless التي تعتمد على التنفيذ عديم الحالة (Stateless) [11].

ولحماية كلمات مرور المستخدمين، يستخدم النظام مكتبة bcryptjs، والتي تعتمد على خوارزمية التجزئة (Hashing) مع إضافة قيمة عشوائية (Salt) لكل كلمة مرور قبل تخزينها داخل قاعدة البيانات. ويؤدي ذلك إلى منع استرجاع كلمات المرور الأصلية حتى في حال الوصول غير المصرح به إلى قاعدة البيانات، كما يزيد من مقاومة النظام لهجمات القوة الغاشمة (Brute Force) وهجمات جداول قوس قزح (Rainbow Tables) [11].

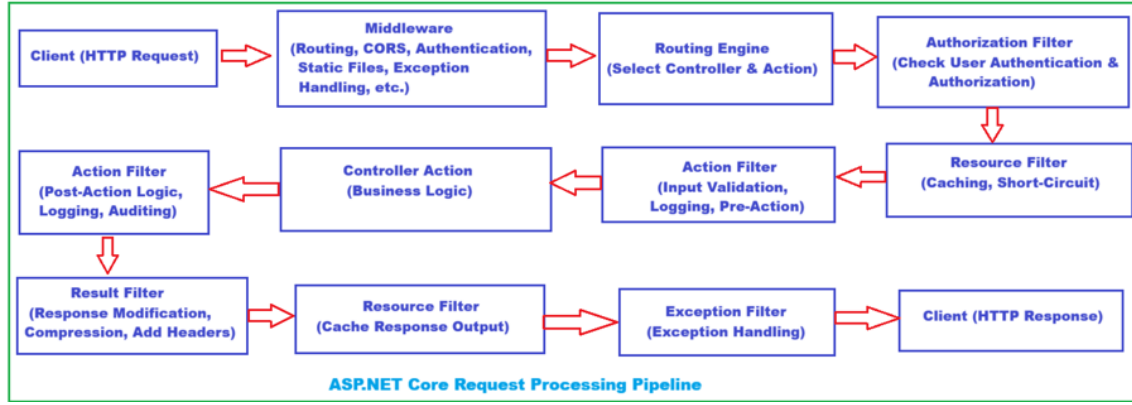
ويخلص الجدول (7-2) الأدوات الرئيسية المستخدمة في تطوير الواجهة الخلفية وأسباب اختيارها.

الجدول 7-2: أدوات تطوير الواجهة الخلفية وإطارات العمل المستخدمة

الفئة	الأداة	الوظيفة داخل النظام	مبرر الاختيار
إطار العمل	Hono	بناء واجهات REST API	خفيف، سريع، ومتوافق مع Cloudflare Workers
التحقق من البيانات	Zod	التحقق من صحة البيانات (Schema Validation)	منع البيانات غير الصحيحة وتحسين موثوقية النظام
المصادقة	JWT	التحقق من هوية المستخدم	مصادقة عديمة الحالة (Stateless Authentication)
حماية كلمات المرور	bcryptjs	تشفير كلمات المرور قبل التخزين	حماية عالية ضد هجمات فك كلمات المرور

يتضح من الجدول (7-2) أن جميع الأدوات المختارة تتوافق مع متطلبات بيئة Serverless، إذ تركز على تقليل استهلاك الموارد، وتحسين سرعة الاستجابة، ورفع مستوى الأمان، مع المحافظة على سهولة تطوير النظام وصيانيته.

كما يعتمد النظام على أسلوب Middleware في معالجة الطلبات، حيث تمر جميع طلبات المستخدم عبر مجموعة من المراحل قبل الوصول إلى منطق التطبيق الأساسي. وتشمل هذه المراحل التحقق من صحة البيانات، والتحقق من هوية المستخدم، وتحديد صلاحيات الوصول، ومعالجة الأخطاء، ثم تنفيذ الوظيفة المطلوبة وإرجاع الاستجابة النهائية. ويساعد هذا الأسلوب في فصل المسؤوليات البرمجية، وتحسين قابلية الصيانة، وإعادة استخدام المكونات البرمجية داخل النظام.



الشكل 2-6: دورة معالجة طلبات واجهة برمجة التطبيقات في الواجهة الخلفية

يتضح من الشكل (2-6) أن طلب المستخدم يمر بعدة مراحل متتابعة تبدأ باستقبال طلب HTTP، ثم المرور بطبقة Middleware المسؤولة عن التوجيه (Routing)، والتحقق من الهوية (Authentication)، والتحقق من صلاحيات الوصول (Authorization)، ثم التحقق من صحة البيانات، قبل تنفيذ منطق التطبيق داخل وحدة المعالجة (Controller). وبعد انتهاء تنفيذ العملية، تُجرى عمليات معالجة الاستجابة والتعامل مع الأخطاء قبل إعادة النتيجة إلى المستخدم. ويسهم هذا التسلسل في تنظيم بنية التطبيق، وتقليل الترابط بين مكوناته، وتحسين أمان واجهات برمجة التطبيقات وموثوقيتها.

3-2-2 محرك التحويل ومعالجة المحتوى (Conversion Engine & Content Processing)

يمثل محرك التحويل العنصر الأساسي في النظام، إذ يتولى معالجة مستندات Markdown وتحويلها إلى مستندات PDF جاهزة للطباعة والعرض. ولا تقتصر هذه العملية على تحويل النصوص فقط، وإنما تشمل سلسلة من مراحل المعالجة (Processing Pipeline) التي تهدف إلى إنتاج مستند متكامل يدعم تنسيق النصوص، والمعادلات الرياضية، والنصوص البرمجية، والمخططات البيانية، مع المحافظة على جودة المخرجات النهائية. لذلك يعتمد المشروع على مجموعة من المكتبات المتخصصة، يؤدي كل منها وظيفة محددة ضمن سلسلة المعالجة [12].

تبدأ عملية التحويل بتحليل مستند Markdown باستخدام مكتبة Marked، والتي تُعد من أشهر المحللات (Parsers) المستخدمة لتحويل ملفات Markdown إلى عناصر HTML. وتمتاز هذه المكتبة بسرعة التنفيذ ودعمها للإضافات (Extensions)، مما يسمح بإجراء تعديلات مخصصة أثناء عملية التحويل بما يتوافق مع متطلبات المشروع [12].

وبالتزامن مع ذلك، تُستخدم مكتبة Gray-matter لاستخراج البيانات الوصفية (Frontmatter) الموجودة في بداية المستند، مثل عنوان المستند، واسم المؤلف، والكلمات المفتاحية، وغيرها من المعلومات التي يمكن استخدامها في إنشاء رأس المستند أو إضافتها إلى بيانات ملف PDF النهائي [13].

أما بالنسبة للنصوص برمجية البرمجية، فيستخدم النظام مكتبة Highlight.js لتطبيق التلوين النحوي (Syntax Highlighting)، حيث تقوم بتحليل لغة البرمجة المستخدمة داخل كتلة النص برمجي وإضافة التنسيق المناسب لكل عنصر، مما يحسن من قابلية قراءة النصوص برمجية داخل المستندات التقنية ويجعلها أكثر وضوحاً للمستخدم [14].

ولعرض المعادلات الرياضية، يعتمد المشروع على مكتبة KaTeX، والتي تقوم بتحويل معادلات LaTeX إلى عناصر HTML و CSS عالية الجودة قبل مرحلة إنشاء ملف PDF. وتمتاز KaTeX بسرعة التنفيذ وانخفاض استهلاكها للموارد مقارنة بمحركات العرض الرياضية الأخرى، الأمر الذي يجعلها مناسبة للعمل ضمن بيئات Serverless [15].

كما يوفر النظام دعماً للمخططات البيانية المكتوبة بلغة PlantUML، حيث تُستخدم مكتبة PlantUML Encoder لتحويل وصف المخطط إلى صيغة يمكن إرسالها إلى خادم PlantUML، والذي يعيد المخطط على هيئة رسم متجهي SVG. ويُدمج هذا الرسم مباشرة داخل مستند HTML قبل إنشاء ملف PDF، مما يضمن المحافظة على جودة المخطط مهما تغير حجم التكبير أو دقة الطباعة [16].

ويبين الجدول (2-8) مكونات محرك التحويل والوظيفة التي تؤديها كل أداة داخل النظام.

الجدول 2-8: مكونات محرك التحويل ومعالجة المحتوى

الوظيفة	المخرجات	المدخلات	الأداة	مرحلة المعالجة
تحويل بنية المستند إلى HTML	HTML	Markdown	Marked	تحليل Markdown
فصل البيانات الوصفية عن المحتوى	Metadata + Content	Markdown + Frontmatter	Gray-matter	استخراج البيانات الوصفية

تطبيق Syntax Highlighting	HTML منسق	كتل النصوص البرمجية	Highlight.js	تلوين النصوص برمجية
عرض المعادلات الرياضية	HTML + CSS	LaTeX	KaTeX	عرض المعادلات
تحويل المخططات إلى رسومات متجهة	Inline SVG	PlantUML	PlantUML Encoder	معالجة المخططات

يتضح من الجدول (2-8) أن عملية التحويل تمر بعدة مراحل مترابطة، حيث تضيف كل أداة تحسناً معيناً على المستند قبل انتقاله إلى المرحلة التالية، مما يؤدي في النهاية إلى إنتاج ملف PDF متكامل يدعم مختلف أنواع المحتوى الفني والعلمي.

وبعد الانتهاء من جميع مراحل المعالجة السابقة، يُرسل مستند HTML الناتج إلى خدمة Cloudflare Browser Rendering، والتي تتولى تحميل الصفحة داخل متصفح Chromium الرأسي وتنفيذ جميع عناصرها الديناميكية قبل إنشاء ملف PDF النهائي، وهو ما يضمن توافق المخرجات مع معايير الويب الحديثة والمحافظة على التخطيط البصري للمستند.

2-2-4 الواجهة الأمامية والتصميم البصري (Frontend & Visual Design)

تمثل الواجهة الأمامية نقطة التفاعل المباشر بين المستخدم والنظام، ولذلك تؤثر بصورة كبيرة في تجربة الاستخدام وسرعة الاستجابة وسهولة الوصول إلى وظائف النظام. وعلى الرغم من الانتشار الواسع لإطارات العمل الحديثة مثل React و Vue و Angular، إلا أن استخدامها في التطبيقات التي تعتمد بصورة رئيسية على معالجة البيانات في الواجهة الخلفية قد يؤدي إلى زيادة حجم الملفات المحملة، وارتفاع زمن تحميل الصفحة، واستهلاك موارد إضافية دون تحقيق فائدة حقيقية. ولهذا السبب يعتمد المشروع على تقنيات الويب الأساسية (Vanilla Web)، والتي تشمل HTML و CSS و JavaScript، لتطوير واجهة مستخدم خفيفة وسريعة تتوافق مع طبيعة بيئة Serverless [17].

يعتمد هذا النهج على تنفيذ جميع عمليات معالجة المستندات داخل الواجهة الخلفية، بينما تقتصر مهام الواجهة الأمامية على استقبال مدخلات المستخدم، وإرسالها إلى واجهة برمجة التطبيقات (REST API)، ثم عرض النتائج النهائية. ويساعد هذا الأسلوب في تقليل حجم الحزمة البرمجية (Bundle Size)، وتحسين زمن التفاعل الأولي (Time to Interactive - TTI)، بالإضافة إلى تبسيط عملية التطوير والصيانة [17].

كما يولي المشروع اهتماماً كبيراً بجودة العرض الطباعي (Typography)، نظراً لأن الهدف النهائي للنظام هو إنتاج مستندات PDF احترافية تدعم اللغتين العربية والإنجليزية. ولتحقيق ذلك، تم اختيار خط Noto Sans Arabic لعرض النصوص العربية، وذلك لما يوفره من دعم كامل لمحارف Unicode وتوافق ممتاز مع اتجاه الكتابة من اليمين إلى اليسار (RTL)، إضافة إلى وضوحه عند العرض على الشاشات والطباعة [17].

أما بالنسبة للنصوص برمجية البرمجية، فقد تم استخدام خط Fira Code، وهو خط أحادي المسافة (Monospace) يتميز بدعمه لخاصية Ligatures، والتي تسمح بدمج بعض الرموز البرمجية المتكررة في أشكال بصرية أكثر وضوحاً، مثل (<=) و(==) و(!=). ويسهم ذلك في تحسين قابلية قراءة النصوص البرمجية وتقليل الإجهاد البصري أثناء مراجعة المستندات التقنية [17].

وبالإضافة إلى اختيار الخطوط المناسبة، تعتمد الواجهة الأمامية على مبادئ التصميم المتجاوب (Responsive Design)، بحيث يمكن استخدامها بسهولة على مختلف أحجام الشاشات، سواء على الحواسيب المكتبية أو الأجهزة اللوحية أو الهواتف الذكية. كما يتم تحميل الخطوط والموارد الثابتة عبر شبكات توزيع المحتوى (Content Delivery Networks - CDNs)، مما يضمن سرعة الوصول إليها وتقليل زمن تحميل الصفحة [17].

ويخلص الجدول (2-9) أهم التقنيات المستخدمة في تطوير الواجهة الأمامية وأسباب اختيارها.

الجدول 2-9: تقنيات الواجهة الأمامية والتصميم البصري

الفئة	التقنية	الاستخدام في المشروع	مبرر الاختيار
بنية الواجهة	HTML5	إنشاء هيكل صفحات النظام	معياري عالمي، بسيط ومتوافق مع جميع المتصفحات
تنسيق الصفحات	CSS3	تصميم الواجهة وتنسيق العناصر	دعم التصميم المتجاوب وإمكانات تنسيق متقدمة

التفاعل البرمجي	JavaScript	تنفيذ العمليات التفاعلية وإرسال الطلبات إلى API	سرعة التنفيذ ودعم جميع المتصفحات الحديثة
الخط العربي	Noto Sans Arabic	عرض النصوص العربية	دعم كامل للعربية واتجاه RTL
خط النصوص البرمجية	Fira Code	عرض النصوص البرمجية	دعم Ligatures وتحسين قابلية قراءة النصوص البرمجية
تحميل الموارد	CDN	تحميل الخطوط والملفات الثابتة	تقليل زمن التحميل وتحسين الأداء

يتضح من الجدول (2-9) أن التقنيات المستخدمة في الواجهة الأمامية اختيرت بما يتوافق مع طبيعة المشروع، حيث تحقق التوازن بين الأداء وسهولة الاستخدام وجودة العرض، مع المحافظة على بساطة البنية البرمجية وتقليل التعقيد الناتج عن استخدام إطارات عمل كبيرة.

ومن خلال الدمج بين واجهة أمامية خفيفة، وبنية خلفية تعمل على الحوسبة الطرفية، ومحرك متكامل لمعالجة المحتوى، يوفر النظام تجربة استخدام سلسة تتيح للمستخدم إنشاء ملفات PDF احترافية بسرعة وكفاءة، مع المحافظة على جودة التخطيط ودقة عرض العناصر المختلفة داخل المستند.

2-3 الأعمال السابقة والدراسات المرتبطة

يُعد استعراض الأعمال السابقة والدراسات المرتبطة خطوة أساسية في تطوير الأنظمة البرمجية، إذ يساهم في فهم الحلول الحالية وتحليل نقاط القوة والقصور فيها، مما يساعد على تحديد الفجوة البحثية التي يسعى المشروع إلى معالجتها. ويستعرض هذا القسم أشهر أدوات تحويل مستندات Markdown إلى PDF، والخدمات السحابية التجارية، إضافة إلى الدراسات الأكاديمية ذات الصلة، قبل تقديم تحليل نقدي يوضح موقع المشروع الحالي مقارنةً بالأعمال السابقة.

2-3-1 أدوات وتقنيات معالجة المستندات الرقمية (Digital Document Processing Tools)

شهدت تقنيات معالجة المستندات الرقمية تطوراً ملحوظاً خلال السنوات الأخيرة، حيث ركزت الأبحاث الحديثة على تحسين آليات تحليل المستندات واستخراج بنيتها المنطقية ومعالجة محتواها بكفاءة عالية. وتؤكد الدراسات أن الاعتماد على صيغ وصفية مثل Markdown و LaTeX يسهم في فصل المحتوى عن التنسيق، مما يسهل إعادة استخدام المستندات وتحويلها إلى صيغ متعددة مع المحافظة على بنيتها ودقتها. كما تشير هذه الدراسات إلى أن دمج مراحل التحليل، واستخراج البيانات الوصفية، ومعالجة العناصر المختلفة ضمن خط معالجة موحد يؤدي إلى تحسين جودة المستندات الرقمية وقابليتها للتبادل بين الأنظمة المختلفة [18].

2-3-2 البنى السحابية الحديثة لمعالجة المستندات (Cloud-Based Document Processing)

اتجهت الأبحاث الحديثة إلى الاستفادة من تقنيات الحوسبة السحابية والحوسبة بدون خوادم (Serverless Computing) في تنفيذ خطوات معالجة المستندات، لما توفره من مرونة عالية في التوسع وتقليل التكاليف التشغيلية. وأظهرت الدراسات أن تنفيذ عمليات معالجة المستندات ضمن بيئات Serverless يسهم في تحسين استغلال الموارد، وتقليل زمن الاستجابة، ورفع كفاءة تنفيذ المهام المعقدة، خاصة عند التعامل مع أعداد كبيرة من طلبات التحويل أو خطوات المعالجة المتوازية [3][3].

2-3-3 الدراسات الأكاديمية المرتبطة (Related Academic Studies)

تناولت العديد من الدراسات الحديثة توظيف الحوسبة بدون خوادم في معالجة البيانات والمستندات الرقمية، حيث ركزت على تحسين أداء خطوات المعالجة (Data Pipelines)، وتقليل استهلاك الموارد، وزيادة قابلية التوسع في البيئات السحابية. كما تناولت أبحاث أخرى تطوير نماذج آمنة لمعالجة البيانات على الحافة (Edge Computing)، مع التركيز على تعزيز الأمان، وتحسين موثوقية الخدمات، وتقليل زمن الاستجابة، مما يجعل هذه التقنيات مناسبة للتطبيقات التي تتطلب إنشاء مستندات رقمية بصورة سريعة وآمنة [3][3].

2-3-4 التحليل النقدي وتحديد الفجوة البحثية (Critical Analysis & Research Gap)

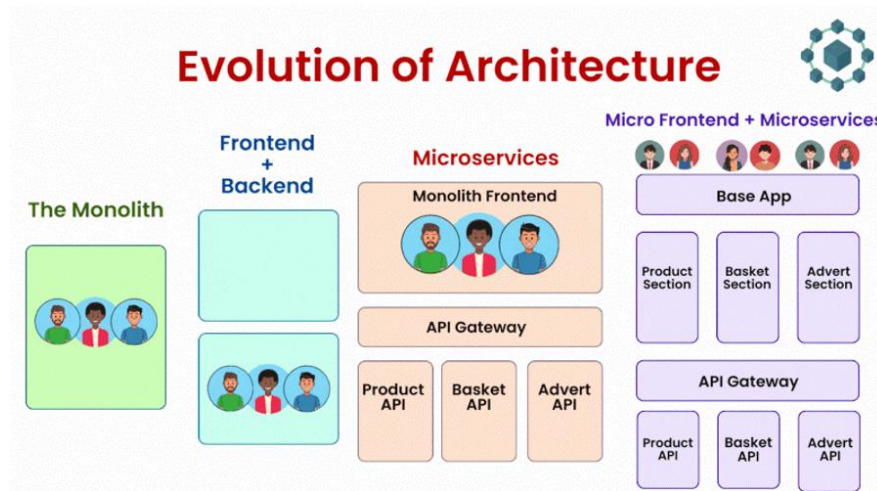
من خلال استعراض الأدوات مفتوحة المصدر، والخدمات السحابية التجارية، والدراسات الأكاديمية، يتضح أن معظم الحلول الحالية تركز على جانب معين من عملية التحويل، دون توفير حل متكامل يجمع بين الأداء المرتفع، والدعم الكامل للغة العربية، ومعالجة العناصر الديناميكية، والعمل ضمن بيئة Serverless Edge.

ويوضح الجدول (10-2) مقارنة بين الفئات الرئيسة للحلول الحالية والنظام المقترح في هذا المشروع.

الجدول 10-2: مقارنة تحليلية بين الأعمال السابقة والنظام المقترح

معيار المقارنة	أدوات مفتوحة المصدر	الخدمات السحابية التجارية	النظام المقترح
البنية التحتية	محلية أو خوادم تقليدية	سحابية مركزية	Serverless Edge
إدارة الخوادم	تقع على عاتق المستخدم	يديرها مزود الخدمة	لا تتطلب إدارة خوادم
دعم اللغة العربية (RTL)	محدود	جزئي	دعم كامل
دعم PlantUML	يحتاج أدوات خارجية	محدود	مدمج بصيغة SVG
دعم KaTeX	يعتمد على LaTeX	محدود	مدمج بالكامل
قابلية التوسع	محدودة	جيدة	توسع تلقائي
التكلفة	تعتمد على الخادم	اشتراكات أو دفع لكل تحويل	منخفضة وفق الاستخدام

يتبين من الجدول (10-2) أن النظام المقترح يجمع بين مزايا الحلول السابقة، مع معالجة معظم القيود التي تعاني منها، سواء من ناحية الأداء أو دعم اللغة العربية أو العناصر الديناميكية.



الشكل 2-7: تطور البنى المعمارية للأنظمة البرمجية وصولاً إلى المعماريات الحديثة

يتضح من الشكل (2-7) أن هندسة البرمجيات تطورت تدريجياً من التطبيقات الأحادية (Monolithic Architecture) إلى المماريات المعتمدة على الخدمات المصغرة (Microservices)، ثم إلى الأنظمة الحديثة التي تجمع بين Micro Frontends و Microservices. ويعكس هذا التطور الاتجاه نحو تصميم أنظمة أكثر مرونة وقابلية للتوسع وسهولة في الصيانة، وهو ما يتوافق مع فلسفة المشروع الحالي، الذي يعتمد على بنية Serverless Edge لتوفير خدمة تحويل مستندات عالية الأداء وقابلة للتوسع دون الحاجة إلى إدارة البنية التحتية التقليدية.

ومن خلال المقارنة السابقة يتضح وجود فجوة بحثية تتمثل في غياب نظام متكامل يجمع بين سهولة الاستخدام، ودعم النصوص العربية ثنائية الاتجاه، ودمج العناصر الديناميكية مثل PlantUML و KaTeX، مع الاستفادة من إمكانيات الحوسبة الطرفية. ويأتي المشروع الحالي لسد هذه الفجوة من خلال تقديم نظام يعتمد على Cloudflare Serverless Edge، ويوفر بيئة متكاملة لتحويل مستندات Markdown إلى PDF بكفاءة عالية وجودة احترافية، مع تقليل الأعباء التشغيلية وتحسين تجربة المستخدم.

الفصل الثالث: تحليل وتصميم النظام

3-1 تحليل النظام

يمثل تحليل النظام المرحلة الأولى في عملية تصميم البرمجيات، حيث يتم تحديد احتياجات المستخدمين والوظائف التي يجب أن يوفرها النظام، إضافة إلى دراسة آلية التفاعل بين المستخدمين ومكونات النظام المختلفة. ويهدف هذا التحليل إلى بناء تصور واضح للمتطلبات الوظيفية (Functional Requirements) وغير الوظيفية (Non-functional Requirements)، بما يضمن تطوير نظام يلبي احتياجات المستخدمين بكفاءة وموثوقية.

في المشروع الحالي يعتمد النظام على نموذج خدمات الويب (Web-based Service)، حيث يتم تنفيذ جميع عمليات معالجة المستندات داخل بيئة Cloudflare Serverless Edge، بينما يتفاعل المستخدم مع النظام من خلال واجهة ويب أو عبر واجهات برمجة التطبيقات (REST API). ولتحقيق ذلك تم تحديد الجهات الفاعلة (Actors) الرئيسة التي تتعامل مع النظام، بالإضافة إلى الوظائف التي يستطيع كل منها تنفيذها.

3-1-1 تحديد الجهات الفاعلة (Actors)

يتفاعل مع النظام ثلاث فئات رئيسة من المستخدمين، تختلف صلاحيات كل منها تبعاً لطبيعة استخدام النظام، وهي:

- **المستخدم المسجل (Registered User):** يمتلك حساباً داخل النظام، ويمكنه تسجيل الدخول، وتحويل المستندات، وإدارة ملفه الشخصي، والاطلاع على سجل عمليات التحويل السابقة.
- **المستخدم الضيف (Guest User):** يستطيع استخدام خدمات التحويل الأساسية دون الحاجة إلى إنشاء حساب، إلا أنه لا يستطيع حفظ السجل أو الاستفادة من الخدمات الشخصية التي يوفرها النظام.
- **النظام الخارجي (External API Consumer):** يمثل التطبيقات أو الأنظمة الأخرى التي تتكامل مع النظام باستخدام واجهة برمجة التطبيقات (REST API)، حيث يمكنها إرسال طلبات التحويل واستلام ملفات PDF بصورة آلية.

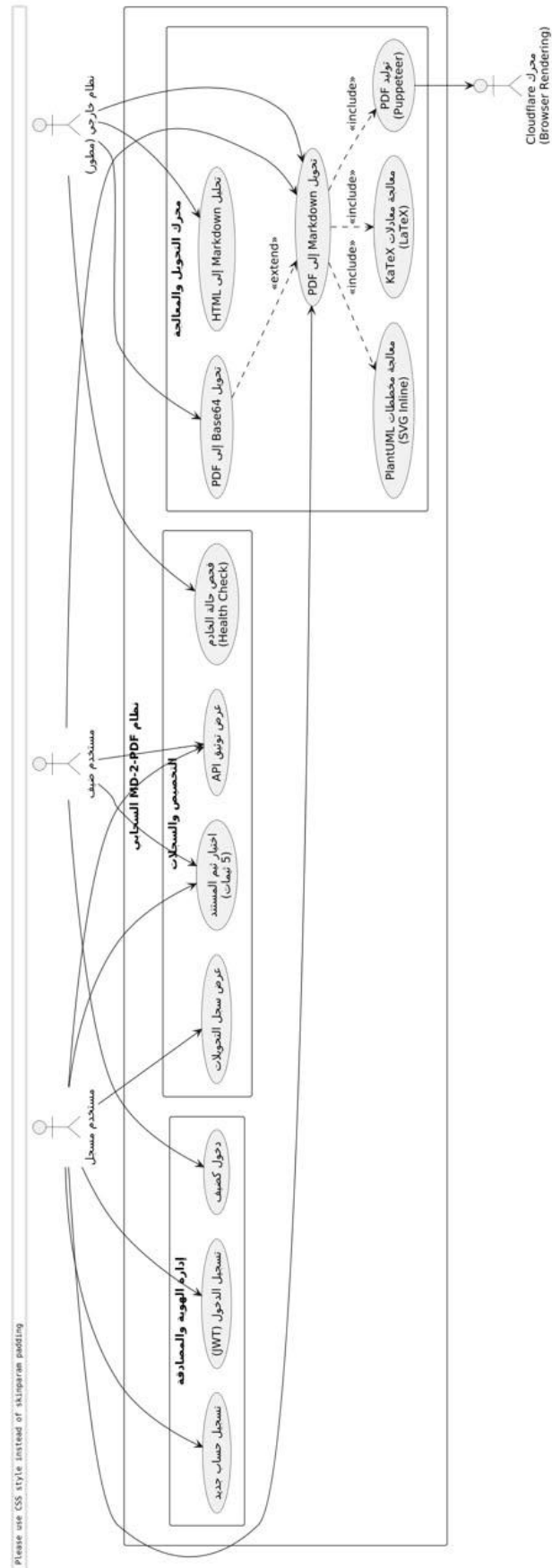
ويمثل تحديد الجهات الفاعلة الأساس لبناء مخطط حالات الاستخدام، إذ يوضح العلاقة بين المستخدمين والوظائف المختلفة التي يقدمها النظام.

3-1-1 مخطط حالات الاستخدام (Use Case Diagram)

يوضح مخطط حالات الاستخدام الوظائف الرئيسية التي يوفرها النظام، والعلاقات بين الجهات الفاعلة المختلفة وبين الخدمات التي يقدمها. ويبين المخطط أن النظام لا يقتصر على عملية تحويل المستندات فقط، وإنما يشمل كذلك إدارة المستخدمين، والمصادقة، وإدارة الإعدادات، وتوفير واجهات برمجية للتكامل مع الأنظمة الخارجية.

وتتوزع وظائف النظام ضمن ثلاث مجموعات رئيسية:

- إدارة المستخدمين والمصادقة، وتشمل إنشاء الحسابات، وتسجيل الدخول باستخدام JWT، وإدارة صلاحيات المستخدمين.
- خدمات التحويل وإدارة المستندات، وتشمل استقبال مستندات Markdown، وتنفيذ عملية التحويل إلى PDF، وإدارة سجل التحويلات.
- الخدمات البرمجية (REST API)، والتي تسمح للأنظمة الخارجية باستخدام محرك التحويل مباشرة دون الحاجة إلى استخدام واجهة الويب.
- كما يوضح المخطط أن عملية تحويل Markdown إلى PDF تعتمد على عدد من العمليات الفرعية، مثل معالجة المعادلات الرياضية باستخدام KaTeX، وتحويل مخططات PlantUML إلى رسومات SVG، ثم تشغيل المتصفح الرأسي Puppeteer لإنتاج ملف PDF النهائي.



الشكل 3-1: مخطط حالات الاستخدام (Use Case Diagram)

يتضح من الشكل (3-1) أن النظام يدعم ثلاثة أنواع من المستخدمين تختلف في مستوى الصلاحيات، كما يبين أن عملية تحويل المستند تمر بعدة عمليات فرعية مترابطة، تبدأ باستقبال مستند Markdown وتنتهي بإنتاج ملف PDF، مع دمج المعادلات الرياضية والمخططات البيانية أثناء عملية التحويل. ويعكس المخطط التكامل بين واجهة المستخدم، وواجهة برمجة التطبيقات، ومحرك التحويل، بما يضمن تقديم خدمة موحدة لجميع فئات المستخدمين.

3-2 تصميم قاعدة البيانات (Database Design)

تُعد قاعدة البيانات أحد المكونات الأساسية في النظام، إذ تتولى تخزين بيانات المستخدمين، وإدارة عمليات المصادقة، وحفظ المستندات وإعدادات التحويل، بالإضافة إلى تسجيل عمليات استخدام واجهات برمجة التطبيقات. وقد تم تصميم قاعدة البيانات باستخدام النموذج العلائقي (Relational Database Model)، مع الاعتماد على قاعدة البيانات Cloudflare D1 المبنية على محرك SQLite، وذلك لما توفره من أداء مرتفع وتكامل كامل مع بيئة Cloudflare Serverless.

وقد روعي أثناء تصميم قاعدة البيانات تطبيق مبادئ التطبيع (Normalization)، بهدف تقليل تكرار البيانات، وتحقيق سلامة العلاقات بين الجداول، وتسهيل عمليات الإضافة والتعديل والاستعلام، مع المحافظة على كفاءة الأداء وقابلية التوسع.

3-2-1 الكيانات الرئيسية (Main Entities)

يتكون مخطط قاعدة البيانات من مجموعة من الكيانات (Entities)، يؤدي كل منها وظيفة محددة ضمن النظام، وهي:

- **Users:** تخزين بيانات المستخدمين الأساسية.
- **Roles:** إدارة صلاحيات المستخدمين.
- **User Profiles:** حفظ البيانات الشخصية للمستخدم.
- **Refresh Tokens:** إدارة جلسات تسجيل الدخول وتجديد رموز المصادقة.
- **Documents:** تخزين المستندات ونتاجات عمليات التحويل.
- **Conversion Settings:** حفظ إعدادات كل عملية تحويل.

- **Themes**: إدارة القوالب والقيمات المستخدمة أثناء إنشاء ملفات PDF.
 - **API Usage Logs**: تسجيل جميع طلبات واجهة برمجة التطبيقات لأغراض المتابعة وتحليل الأداء.
- ويؤدي تقسيم البيانات إلى هذه الجداول إلى زيادة مرونة النظام، وتسهيل عمليات الصيانة والتطوير مستقبلاً.

2-2-3 العلاقات بين الجداول (Relationships)

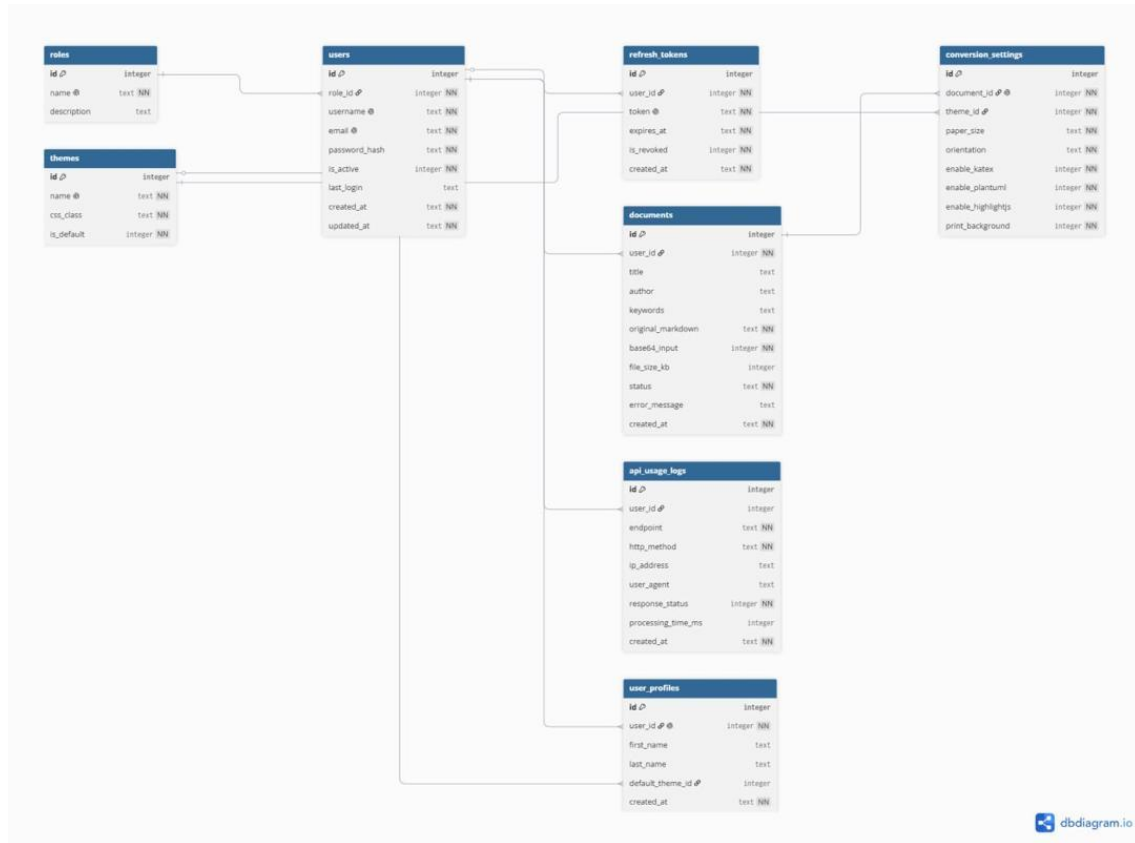
تعتمد قاعدة البيانات على مجموعة من العلاقات المنطقية التي تربط الكيانات المختلفة، ومن أهمها:

- يرتبط جدول Roles مع جدول Users بعلاقة واحد إلى متعدد (One-to-Many)، حيث يمكن لكل دور (Role) أن يرتبط بعدد كبير من المستخدمين، بينما يمتلك كل مستخدم دوراً واحداً فقط.
- يرتبط جدول Users مع جدول User Profiles بعلاقة واحد إلى واحد (One-to-One)، بحيث يمتلك كل مستخدم ملفاً شخصياً واحداً.
- يرتبط جدول Users مع جدول Documents بعلاقة واحد إلى متعدد، إذ يستطيع المستخدم إنشاء عدد غير محدود من المستندات.
- يرتبط جدول Documents مع جدول Conversion Settings بعلاقة واحد إلى واحد، بحيث يمتلك كل مستند إعدادات تحويل مستقلة.
- يرتبط جدول Users مع جدول Refresh Tokens بعلاقة واحد إلى متعدد، مما يسمح بوجود أكثر من جلسة تسجيل دخول للمستخدم نفسه.
- يرتبط جدول Users مع جدول API Usage Logs بعلاقة واحد إلى متعدد، لتسجيل جميع الطلبات التي ينفذها المستخدم داخل النظام.

كما يرتبط جدول Themes مع إعدادات التحويل، بحيث يمكن استخدام أكثر من مستند للقالب نفسه دون الحاجة إلى تكرار بيانات القالب.

3-2-3 تصميم مخطط الكيانات والعلاقات (Entity Relationship Diagram)

يوضح مخطط الكيانات والعلاقات (ERD) البنية المنطقية لقاعدة البيانات والعلاقات التي تربط الجداول المختلفة داخل النظام.



الشكل 3-2: مخطط الكيانات والعلاقات (Entity Relationship Diagram)

يتضح من الشكل (3-2) أن جدول Users يمثل المحور الرئيس لقاعدة البيانات، حيث ترتبط به معظم الجداول الأخرى مثل Documents، وRefresh Tokens، وUser Profiles، وAPI Usage Logs. كما يوضح المخطط أن إعدادات التحويل (Conversion Settings) تُخزن بصورة مستقلة عن المستندات، مما يسمح بتغيير إعدادات كل عملية تحويل دون التأثير على بيانات المستند نفسه. ويظهر المخطط كذلك وجود جداول مستقلة لإدارة الأدوار (Roles) والThemes، وهو ما يعزز قابلية النظام للتوسع وإضافة خصائص جديدة مستقبلاً دون الحاجة إلى إعادة تصميم قاعدة البيانات.

3-2-4 خصائص التصميم

راعى تصميم قاعدة البيانات مجموعة من المبادئ الهندسية، من أهمها:

- تقليل تكرار البيانات من خلال تطبيق مبادئ التطبيق.
- استخدام المفاتيح الأساسية (Primary Keys) والمفاتيح الأجنبية (Foreign Keys) لضمان سلامة العلاقات.
- فصل بيانات المستخدمين عن بيانات المستندات وإعدادات التحويل.
- دعم التوسع المستقبلي بإضافة جداول أو خصائص جديدة دون التأثير على البنية الحالية.
- تحسين سرعة عمليات البحث والاستعلام من خلال تنظيم العلاقات بين الجداول.
- توفير سجل مستقل لطلبات واجهات برمجة التطبيقات لمراقبة الأداء وتحليل الاستخدام.

ويؤدي هذا التصميم إلى توفير قاعدة بيانات مرنة وقابلة للصيانة، تدعم جميع الوظائف الرئيسة للنظام، وتتكامل بصورة مباشرة مع بيئة Cloudflare D1، بما يحقق الأداء المطلوب في تطبيقات Serverless.

3-3 التصميم المعماري للنظام (System Architecture)

يُعد التصميم المعماري (System Architecture) المرحلة التي تُحوّل المتطلبات الوظيفية إلى بنية برمجية واضحة توضح كيفية توزيع مكونات النظام وآلية تفاعلها مع بعضها البعض. ويهدف التصميم المعماري إلى تحقيق الفصل بين المسؤوليات (Separation of Concerns)، وتقليل الترابط بين المكونات (Loose Coupling)، وزيادة قابلية الصيانة والتوسع، بما يضمن سهولة تطوير النظام وإضافة وظائف جديدة مستقبلاً دون التأثير على بقية مكوناته.

يعتمد النظام المقترح على معمارية متعددة الطبقات (Multi-Tier Architecture)، حيث يتم فصل واجهة المستخدم عن منطق التطبيق ومحرك التحويل والخدمات السحابية، مع الاعتماد على بيئة Cloudflare Serverless Edge لتشغيل جميع العمليات دون الحاجة إلى خوادم تقليدية. ويؤدي هذا الفصل إلى تحسين الأداء، وزيادة موثوقية النظام، وتقليل التعقيد البرمجي.

3-3-1 طبقة العميل (Client Layer)

تمثل طبقة العميل نقطة التفاعل المباشر مع النظام، وتتكون من مكونين رئيسيين هما:

- **واجهة الويب (Web Interface):** تتيح للمستخدمين إدخال مستندات Markdown، وتحديد إعدادات التحويل، واستلام ملفات PDF الناتجة من خلال متصفح الويب.
- **الأنظمة الخارجية (REST API Consumers):** وتشمل التطبيقات أو الأنظمة الأخرى التي تستخدم واجهات برمجة التطبيقات (REST API) لإرسال طلبات التحويل واستقبال النتائج بصورة آلية دون الحاجة إلى استخدام واجهة الويب.

تقوم هذه الطبقة بإرسال طلبات HTTP إلى الواجهة الخلفية واستقبال النتائج النهائية فقط، بينما تتم جميع عمليات المعالجة داخل البيئة السحابية.

3-3-2 طبقة الواجهة الخلفية (Application Layer)

تمثل الواجهة الخلفية مركز التحكم الرئيسي في النظام، وقد تم تطويرها باستخدام إطار العمل Hono، حيث تستقبل جميع طلبات المستخدمين، ثم تنفذ سلسلة من عمليات المعالجة قبل تمرير الطلب إلى محرك التحويل. وتشمل أهم وظائف هذه الطبقة:

- التحقق من هوية المستخدم باستخدام JWT.
- تشفير كلمات المرور باستخدام bcryptjs.
- التحقق من صحة البيانات المدخلة باستخدام Zod.
- إدارة الطلبات المرسل إلى محرك التحويل.
- تسجيل عمليات التحويل داخل قاعدة البيانات.
- إعادة النتائج النهائية إلى المستخدم أو إلى واجهات برمجة التطبيقات.

ويضمن هذا التصميم فصل منطق الأعمال (Business Logic) عن بقية مكونات النظام، مما يسهل تطوير النظام وصيانته.

3-3-3 محرك التحويل (Conversion Engine)

يُعد محرك التحويل المكون الأساسي المسؤول عن تحويل مستندات Markdown إلى ملفات PDF، حيث يمر المستند بعدة مراحل معالجة متتابعة قبل الوصول إلى النتيجة النهائية.

وتشمل مراحل المعالجة:

- تحليل مستند Markdown واستخراج البيانات الوصفية (Frontmatter).
- تحويل Markdown إلى HTML باستخدام مكتبة Marked.
- تلوين النصوص البرمجية باستخدام Highlight.js.
- تحويل المعادلات الرياضية بواسطة KaTeX.
- استخراج مخططات PlantUML وتحويلها إلى رسومات SVG.
- دمج جميع العناصر داخل مستند HTML موحد.

وتنتج هذه المرحلة صفحة HTML جاهزة للإرسال إلى خدمة إنشاء ملفات PDF.

3-3-4 الخدمات السحابية والخارجية (Cloud & External Services)

يعتمد النظام على مجموعة من الخدمات الخارجية المتكاملة مع منصة Cloudflare، وتشمل:

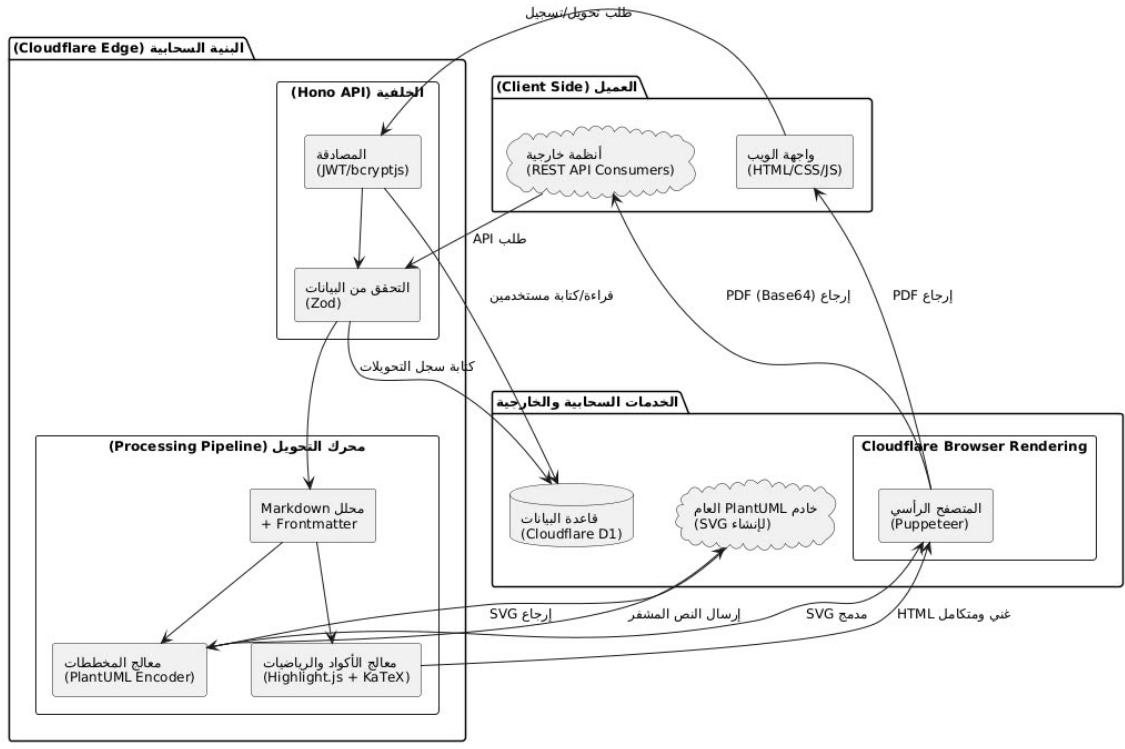
- Cloudflare Browser Rendering لتشغيل متصفح Chromium الرأسي باستخدام Puppeteer.
- Cloudflare D1 لتخزين بيانات المستخدمين وسجل عمليات التحويل.
- PlantUML Server لتوليد الرسوميات المتجهة بصيغة SVG.

وتعمل هذه الخدمات بصورة متزامنة مع الواجهة الخلفية، بحيث يتم تبادل البيانات بينها وفق بروتوكولات HTTP بصورة آمنة وسريعة.

3-3-5 تدفق البيانات بين طبقات النظام

تمر البيانات داخل النظام عبر مجموعة من المراحل تبدأ من المستخدم وتنتهي بإرجاع ملف PDF النهائي، ويمكن تلخيصها كما يلي:

- يرسل المستخدم مستند Markdown إلى النظام.
- تستقبل واجهة Hono الطلب وتتحقق من الهوية وصحة البيانات.
- يُرسل المستند إلى محرك التحويل لمعالجة جميع عناصره.
- تُرسل مخططات PlantUML إلى الخادم المختص لإرجاع ملفات SVG.
- تُدمج الرسوميات والمعادلات والنصوص البرمجية داخل مستند HTML.
- يُرسل مستند HTML إلى خدمة Cloudflare Browser Rendering.
- يقوم المتصفح الرأسي بتحميل الصفحة وإنشاء ملف PDF.
- يُسجل الطلب داخل قاعدة بيانات Cloudflare D1.
- يُعاد ملف PDF إلى المستخدم أو إلى التطبيق الخارجي.



الشكل 3-3: التصميم المعماري للنظام (System Architecture)

يتضح من الشكل (3-3) أن النظام يعتمد على معمارية متعددة الطبقات، تبدأ بطبقة العميل، ثم طبقة الواجهة الخلفية، يليها محرك التحويل، ثم الخدمات السحابية الخارجية. كما يوضح المخطط أن جميع عمليات معالجة المستندات تتم داخل بيئة Cloudflare Edge، بينما تتولى خدمة Browser Rendering إنشاء ملف PDF النهائي، في حين تستخدم قاعدة البيانات Cloudflare D1 لتخزين بيانات المستخدمين وسجل التحويلات. ويظهر المخطط كذلك آلية التكامل مع خادم PlantUML لتوليد الرسومات المتجهة، الأمر الذي يحقق فصلاً واضحاً بين المسؤوليات ويسهم في تحسين قابلية النظام للتوسع والصيانة.

3-3-6 مزايا التصميم المعماري

يحقق التصميم المعماري المعتمد في المشروع مجموعة من المزايا، من أبرزها:

- الفصل بين طبقات النظام وتقليل الترابط بينها.

- دعم التوسع التلقائي بفضل بيئة Cloudflare Serverless.
- سهولة صيانة النظام وإضافة وظائف جديدة.
- تحسين الأداء وتقليل زمن الاستجابة من خلال الحوسبة الطرفية.
- توفير درجة عالية من الأمان عبر التحقق من الهوية وصحة البيانات.
- إمكانية دمج النظام مع تطبيقات خارجية باستخدام واجهات REST API.
- الاعتماد على خدمات سحابية مُدارة بالكامل دون الحاجة إلى إدارة الخوادم.

3-4 سير عمل النظام (System Workflow)

يمثل سير العمل (Workflow) التسلسل المنطقي للعمليات التي ينفذها النظام منذ لحظة استلام طلب المستخدم وحتى إنتاج ملف PDF النهائي. ويهدف هذا التسلسل إلى تنظيم مراحل معالجة البيانات، وضمان تنفيذ جميع عمليات التحقق والمعالجة بصورة صحيحة قبل إنشاء المستند النهائي، مما يساهم في تحسين موثوقية النظام وتقليل احتمالية حدوث الأخطاء أثناء التنفيذ.

يعتمد النظام على سلسلة متتابعة من العمليات تبدأ بالتحقق من صحة الطلب وهوية المستخدم، ثم معالجة محتوى مستند Markdown، وإضافة العناصر الديناميكية، وانتهاءً بتوليد ملف PDF وإعادته إلى المستخدم. وقد صُممت هذه المراحل بحيث تعمل داخل بيئة Cloudflare Serverless Edge، مع الاستفادة من الخدمات السحابية الخارجية عند الحاجة، مثل خدمة PlantUML وخدمة Cloudflare Browser Rendering.

3-4-1 استقبال الطلب والتحقق من صحته

تبدأ عملية التحويل عند استلام طلب من واجهة الويب أو من إحدى واجهات برمجة التطبيقات (REST API). ويحتوي الطلب على مستند Markdown بالإضافة إلى إعدادات التحويل المطلوبة.

وفي البداية، يقوم النظام بتنفيذ مرحلتين أساسيتين:

- التحقق من هوية المستخدم باستخدام JWT في حال كان الطلب يتطلب المصادقة.
- التحقق من صحة جميع البيانات المدخلة باستخدام مكتبة Zod، للتأكد من اكتمال الحقول وصحة القيم قبل بدء عملية المعالجة.

وفي حال اكتشاف أي خطأ في البيانات، يتم إيقاف عملية التحويل وإرجاع رسالة خطأ مناسبة دون متابعة بقية المراحل.

3-4-2 معالجة محتوى المستند

بعد اجتياز مرحلة التحقق، تبدأ عملية معالجة محتوى المستند من خلال محرك التحويل، والتي تتضمن مجموعة من الخطوات المتتابعة، هي:

- استخراج البيانات الوصفية (Frontmatter) باستخدام Gray-matter.
- تحويل مستند Markdown إلى HTML بواسطة Marked.
- تطبيق تلوين النصوص البرمجية باستخدام Highlight.js.
- تفسير المعادلات الرياضية بواسطة KaTeX.
- استخراج أوامر PlantUML الموجودة داخل المستند.

وتهدف هذه المرحلة إلى تحويل جميع عناصر المستند إلى مكونات HTML جاهزة للعرض داخل المتصفح.

3-4-3 معالجة المخططات البيانية

عند اكتشاف أي مخطط مكتوب بلغة PlantUML، يقوم النظام بإرسال نص المخطط إلى خادم PlantUML، والذي يتولى إنشاء الرسم البياني بصيغة SVG.

وبعد استلام الرسم، يتم دمجها مباشرة داخل صفحة HTML باستخدام أسلوب Inline SVG، مما يحافظ على الجودة العالية للرسومات عند الطباعة أو التكبير، ويمنع فقدان الدقة الذي يحدث عند استخدام الصور النقطية.

3-4-4 إنشاء ملف PDF

بعد اكتمال جميع عمليات المعالجة، تصبح صفحة HTML جاهزة للإرسال إلى خدمة Cloudflare Browser Rendering.

ويقوم المتصفح الرأسي Chromium بتنفيذ الخطوات التالية:

- تحميل صفحة HTML.
- تطبيق جميع ملفات CSS والخطوط.
- تنفيذ عناصر JavaScript إن وجدت.
- عرض المعادلات الرياضية والمخططات البيانية.
- تطبيق إعدادات الطباعة الخاصة بالمستند.
- إنشاء ملف PDF النهائي.

ويضمن هذا الأسلوب إنتاج مستند مطابق تماماً لما يظهر داخل المتصفح، مع المحافظة على جودة التخطيط والتنسيق.

3-4-5 تسجيل العملية وإرجاع النتيجة

بعد انتهاء عملية إنشاء ملف PDF، يقوم النظام بتسجيل بيانات عملية التحويل داخل قاعدة بيانات Cloudflare D1، بما يشمل وقت التنفيذ، وحالة العملية، وبعض المعلومات الإحصائية اللازمة لمتابعة أداء النظام.

وفي النهاية، يتم إرسال ملف PDF الناتج إلى المستخدم أو إلى التطبيق الخارجي الذي أرسل الطلب، سواء على شكل ملف مباشر أو على هيئة بيانات Base64 وفقاً لنوع واجهة برمجة التطبيقات المستخدمة.



الشكل 3-4: مخطط سير عمل النظام لتحويل Markdown إلى PDF

يتضح من الشكل (3-4) أن عملية التحويل تمر بمجموعة من المراحل المتتابعة تبدأ باستقبال الطلب والتحقق من هوية المستخدم وصحة البيانات، ثم معالجة مستند Markdown واستخراج مكوناته المختلفة، بما في ذلك المعادلات الرياضية ومخططات PlantUML، يلي ذلك دمج جميع العناصر داخل مستند HTML وإرساله إلى خدمة Cloudflare Browser Rendering لإنشاء ملف PDF النهائي. وفي حال نجاح العملية يتم تسجيلها في قاعدة بيانات Cloudflare D1 وإرجاع الملف إلى المستخدم، بينما يتم التعامل مع الأخطاء وإرجاع الرسائل المناسبة في حال فشل أي مرحلة من مراحل المعالجة.

الفصل الرابع: تنفيذ النظام والتطبيق

العملي

الواجهة الرئيسية للنظام

تمثل الواجهة الرئيسية نقطة الدخول الأولى للمستخدم، وقد صُممت لتكون بسيطة وسهلة الاستخدام مع المحافظة على مظهر حديث ومتوافق مع الهوية البصرية للمشروع. كما تدعم الواجهة اللغتين العربية والإنجليزية، مع إمكانية التبديل بين اتجاهي الكتابة RTL و LTR بما يتناسب مع لغة المستندات المراد تحويلها.

تتضمن الصفحة الرئيسية مجموعة من الوظائف الأساسية التي يحتاجها المستخدم قبل البدء بعملية التحويل، مثل اختيار اللغة، وتحديد اتجاه النص، والوصول إلى توثيق واجهات برمجة التطبيقات، بالإضافة إلى إمكانية استعراض سجل التحويلات السابقة للمستخدمين المسجلين.

1-1-4 الصفحة الرئيسية للنظام

تظهر الصفحة الرئيسية مباشرة بعد دخول المستخدم إلى الموقع، حيث تعرض عنوان النظام، ووصفاً مختصراً لوظيفته، بالإضافة إلى عناصر التحكم الأساسية اللازمة لبدء عملية التحويل.

وتوفر الصفحة عدداً من الخيارات، من أهمها:

- تسجيل الدخول أو استخدام النظام كضيف.
- اختيار لغة واجهة الاستخدام.
- تحديد اتجاه النص (RTL أو LTR).
- الوصول إلى سجل التحويلات.
- استعراض توثيق واجهات برمجة التطبيقات (API Documentation).

وقد روعي في تصميم الصفحة اعتماد واجهة بسيطة تقلل عدد الخطوات المطلوبة من المستخدم، مما يسهم في تحسين تجربة الاستخدام وسهولة الوصول إلى الوظائف المختلفة.



الشكل 4-1: الواجهة الرئيسية لنظام تحويل PDF إلى Markdown

يتضح من الشكل (4-1) أن الصفحة الرئيسية تعرض جميع الوظائف الأساسية بصورة منظمة، حيث يظهر عنوان النظام في منتصف الصفحة، بينما تتوزع أدوات التحكم في أعلى الواجهة، مثل اختيار اللغة، وتحديد اتجاه الكتابة، والدخول إلى سجل التحويلات أو توثيق واجهة برمجة التطبيقات، مما يوفر نقطة انطلاق موحدة لجميع مستخدمي النظام.

4-1-2 تسجيل الدخول وإنشاء الحساب

يوفر النظام آلية مصادقة مرنة تسمح باستخدامه بطريقتين مختلفتين، الأولى من خلال إنشاء حساب وتسجيل الدخول للاستفادة من جميع خصائص النظام، والثانية عبر استخدام النظام كضيف دون الحاجة إلى إنشاء حساب.

عند اختيار تسجيل الدخول تظهر نافذة تحتوي على تبويبين، أحدهما لإنشاء حساب جديد، والآخر لتسجيل الدخول باستخدام البريد الإلكتروني وكلمة المرور. كما يمكن للمستخدم اختيار وضع الضيف، والذي يسمح بتنفيذ عمليات التحويل مع الاحتفاظ بسجل العمليات داخل جلسة التصفح الحالية فقط.

ويعتمد النظام على بروتوكول JWT (JSON Web Token) لإدارة جلسات المستخدمين بعد نجاح عملية المصادقة، بينما يتم تشفير كلمات المرور باستخدام مكتبة bcryptjs لضمان حماية بيانات المستخدمين.

الشكل 4-2: نافذة تسجيل الدخول وإنشاء الحساب

يتضح من الشكل (4-2) أن النظام يوفر أكثر من أسلوب للوصول إلى خدماته، حيث يمكن للمستخدم إنشاء حساب جديد أو تسجيل الدخول باستخدام بياناته السابقة، كما يتيح خيار المتابعة كضيف، وهو ما يمنح المستخدم مرونة في استخدام النظام وفق احتياجاته، مع المحافظة على مستوى مناسب من الأمان وإدارة الجلسات.

4-2 واجهة تحويل Markdown إلى PDF

تمثل واجهة التحويل المكون الرئيس في النظام، حيث توفر للمستخدم جميع الأدوات اللازمة لإنشاء مستندات Markdown وتحويلها إلى ملفات PDF بصورة مباشرة. وقد روعي في تصميم هذه الواجهة توفير بيئة عمل متكاملة تجمع بين إدخال النصوص، والمعاينة الفورية، وإدارة الملفات، مع المحافظة على سهولة الاستخدام وسرعة الاستجابة.

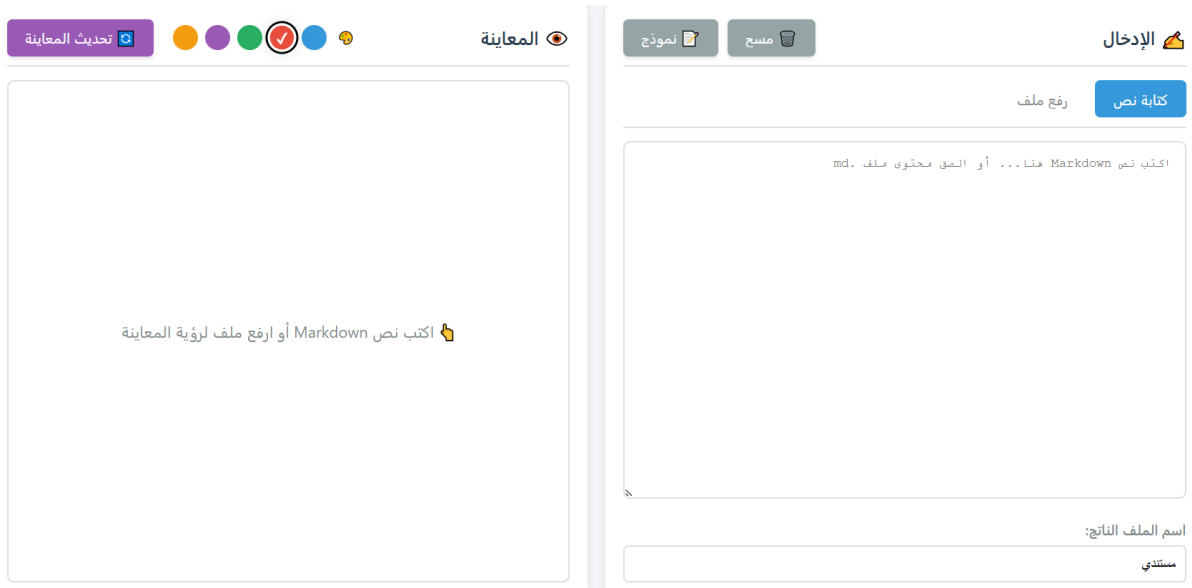
وتنقسم الواجهة إلى جزأين رئيسيين؛ الأول مخصص لإدخال النصوص أو رفع ملفات Markdown، والثاني لعرض المعاينة المباشرة للمستند بعد معالجته، الأمر الذي يسمح للمستخدم بمراجعة النتيجة قبل تنفيذ عملية التحويل النهائية.

4-2-1 إدخال النصوص والمعاينة المباشرة

يوفر النظام محرراً مخصصاً لكتابة مستندات Markdown، حيث يستطيع المستخدم كتابة النصوص مباشرة داخل المحرر أو تعديل محتوى مستند موجود مسبقاً. وفي المقابل، تعرض نافذة المعاينة النتيجة النهائية بعد تحويل النص إلى HTML بصورة فورية، مع تطبيق جميع التنسيق المدعومة. وتتضمن هذه الواجهة مجموعة من الأدوات المساعدة، مثل:

- إنشاء مستند جديد.
- حذف محتوى المحرر.
- اختيار أحد القوالب الجاهزة.
- تغيير ألوان السمة (Theme).
- تحديث نافذة المعاينة مباشرة بعد أي تعديل

وتعتمد المعاينة على محرك التحويل نفسه المستخدم أثناء إنشاء ملف PDF، لذلك فإن النتيجة الظاهرة للمستخدم تكون مطابقة تقريباً للمستند النهائي.



الشكل 4-3: واجهة إدخال Markdown والمعاينة المباشرة

يتضح من الشكل (4-3) أن الواجهة مقسمة إلى منطقتين رئيسيتين؛ المنطقة اليمنى مخصصة لتحرير مستند Markdown وإدخال البيانات، بينما تعرض المنطقة اليسرى المعاينة الفورية للمستند بعد تطبيق جميع التنسيقات، الأمر الذي يتيح للمستخدم التحقق من مظهر المستند قبل البدء بعملية التحويل.

2-2-4 رفع ملفات Markdown

إضافة إلى الكتابة المباشرة، يدعم النظام رفع ملفات (md) Markdown من جهاز المستخدم، مما يختصر الوقت اللازم لإعادة كتابة المستندات الموجودة مسبقاً.

وتسمح واجهة رفع الملفات للمستخدم باختيار ملف واحد أو عدة ملفات، كما تدعم خاصية السحب والإفلات (Drag & Drop)، حيث يتم تحميل الملفات مباشرة داخل المحرر استعداداً لمعالجتها.

وبعد انتهاء عملية الرفع، يقوم النظام بقراءة محتوى الملف تلقائياً وعرضه داخل محرر Markdown مع تحديث نافذة المعاينة بصورة فورية.



الشكل 4-4: واجهة رفع ملفات Markdown

يتضح من الشكل (4-4) أن النظام يوفر أكثر من أسلوب لاختيار الملفات، سواء باستخدام مستعرض الملفات أو من خلال السحب والإفلات، وهو ما يسهل عملية التعامل مع المستندات ويزيد من مرونة استخدام النظام.

4-2-3 تنفيذ عملية التحويل

بعد الانتهاء من كتابة المستند أو رفعه، يستطيع المستخدم بدء عملية التحويل بالضغط على زر "تحويل إلى PDF". وعند تنفيذ هذه العملية، يرسل النظام محتوى المستند إلى الواجهة الخلفية، حيث تبدأ سلسلة من عمليات المعالجة تشمل:

- تحليل مستند Markdown.
- معالجة البيانات الوصفية (Frontmatter).
- تفسير المعادلات الرياضية باستخدام KaTeX.
- تلوين النصوص البرمجية بواسطة Highlight.js.

- معالجة مخططات PlantUML وتحويلها إلى رسومات SVG.
 - إنشاء صفحة HTML النهائية.
 - إرسال الصفحة إلى خدمة Cloudflare Browser Rendering لتوليد ملف PDF.
- وفي حال نجاح العملية، يتم تنزيل الملف مباشرة أو إرجاعه إلى التطبيق المستفيد في حالة استخدام واجهة برمجة التطبيقات.



الشكل 4-5: تنفيذ عملية تحويل Markdown إلى PDF

يتضح من الشكل (4-5) أن عملية التحويل تتم بخطوة واحدة فقط، حيث يقوم المستخدم بالضغط على زر التحويل، بينما يتولى النظام تنفيذ جميع مراحل المعالجة داخلياً بصورة تلقائية، مما يبسط تجربة الاستخدام ويخفي التعقيد البرمجي عن المستخدم النهائي.

3-4 دليل استخدام Markdown

حرصاً على تسهيل استخدام النظام للمستخدمين الجدد، تم تضمين دليل مختصر يوضح أشهر أوامر لغة Markdown داخل الواجهة نفسها، بحيث يستطيع المستخدم الرجوع إليه أثناء كتابة المستند دون الحاجة إلى البحث في مصادر خارجية. ويتضمن الدليل أمثلة عملية على كتابة العناوين، والقوائم، والنصوص العريضة والمائلة، والروابط، والنصوص البرمجية، وغيرها من عناصر التنسيق الأساسية التي تدعمها لغة Markdown.

دليل Markdown السريع
تنسيقات أساسية:
عنوان 1 - عنوان كبير
عنوان 2 - عنوان متوسط
نص عريض - نص عريض
نص مائل - نص مائل
- عنصر في القائمة - قائمة نقطية
[رابط](URL) - رابط
`كود` - كود مضمن

الشكل 4-6: دليل Markdown السريع

يتضح من الشكل (4-6) أن النظام يعرض مرجعاً سريعاً لأهم أوامر Markdown داخل واجهة الاستخدام، مما يساعد المستخدم على كتابة المستندات بصورة صحيحة، ويقلل الحاجة إلى الرجوع إلى مراجع خارجية أثناء العمل.

4-4 توثيق واجهة برمجة التطبيقات (API Documentation)

إلى جانب واجهة الاستخدام الرسومية، يوفر النظام توثيقاً متكاملاً لواجهة برمجة التطبيقات (REST API)، يتيح للمطورين التعرف على جميع الخدمات التي يقدمها النظام وكيفية استخدامها داخل تطبيقاتهم المختلفة. وقد تم تصميم هذا التوثيق ليكون تفاعلياً وسهل الاستخدام، بحيث يعرض تفاصيل نقاط النهاية (Endpoints)، والمعاملات المطلوبة، وأنواع البيانات، والاستجابات المتوقعة لكل خدمة.

ويُعد هذا التوثيق مرجعاً عملياً للمطورين الراغبين في دمج خدمة تحويل Markdown إلى PDF داخل أنظمتهم، دون الحاجة إلى دراسة تفاصيل تنفيذ النظام الداخلية.

4-4-1 الصفحة الرئيسية لتوثيق API

تعرض الصفحة الرئيسية لتوثيق واجهة برمجة التطبيقات معلومات عامة عن النظام، مثل رقم الإصدار، والمنصة المستخدمة، وتنسيق البيانات المدعوم، والبروتوكولات المستخدمة في الاتصال، إضافة إلى رابط المستودع البرمجي.

كما تستعرض الصفحة الخصائص الرئيسة التي يدعمها النظام، مثل دعم اللغة العربية، والمعادلات الرياضية، وتلوين النصوص البرمجية، ومخططات PlantUML، مع تقديم وصف مختصر لكل ميزة.

توثيق API - MD-2-PDF v2.0

نظرة عامة

MD-2-PDF هي أداة سحابية لتحويل نصوص Markdown إلى ملفات PDF احترافية مع دعم كامل للغة العربية (RTL)، المعادلات الرياضية (LaTeX)، تلوين الأكواد، ومخططات PlantUML.

الرابط الأساسي: <https://md-2-pdf.pages.dev>

معلومات سريعة

الإصدار	2.0.0
المنصة	Cloudflare Pages + Workers
تنسيق الطلبات	JSON (<code>Content-Type: application/json</code>)
البروتوكول	HTTPS فقط
رمز المصدر	GitHub Repository

الشكل 4-7: الصفحة الرئيسية لتوثيق واجهة برمجة التطبيقات

يتضح من الشكل (4-7) أن صفحة التوثيق تعرض معلومات تعريفية شاملة حول النظام، إلى جانب قائمة بالميزات المدعومة، مما يساعد المطورين على التعرف على إمكانيات النظام قبل البدء باستخدام واجهات برمجة التطبيقات. كما تعرض الصفحة نفسها أبرز الإمكانيات التي يدعمها النظام، حيث يتم تقديمها على شكل بطاقات توضيحية تتضمن أهم التقنيات المتوفرة، مما يمنح المطور نظرة سريعة على قدرات النظام قبل استخدام واجهات برمجة التطبيقات.

المميزات

 PlantUML مخططات تسلسلية، فئات، وتدفق مباشرة في المستند	 تلوين الأكواد Syntax Highlighting لـ JavaScript, Python, CSS, SQL وأكثر	 LaTeX معادلات رياضية سطرية وعرضية (x^2، $\\$...$\\$)	 دعم RTL دعم كامل للغة العربية والاتجاه من اليمين لليسار
 MCP أدوات للتكامل مع Claude, Gemini, Codex وأكثر	 Frontmatter ليانات YAML Frontmatter المستند الوصفية	 صفحة غلاف صفحة غلاف تلقائية مع عنوان، مؤلف، وتاريخ	 ثيمات متعددة 5 ثيمات: أزرق، أحمر، أخضر، بنفسجي، ذهبي

الشكل 4-8: الميزات المدعومة في النظام

يتضح من الشكل (4-8) أن صفحة التوثيق تعرض أهم الميزات التي يدعمها النظام، مثل دعم اللغة العربية والاتجاه RTL، ومعالجة المعادلات الرياضية باستخدام LaTeX، وتلوين النصوص البرمجية، ودعم مخططات PlantUML، وبيانات Frontmatter، وإنشاء صفحة غلاف، وتعدد السمات، إضافة إلى التكامل مع بروتوكول MCP، مما يوضح الإمكانيات التقنية التي يوفرها النظام للمطورين.

4-4-2 واجهات المصادقة (Authentication APIs)

يوفر النظام مجموعة من واجهات برمجة التطبيقات الخاصة بإدارة المستخدمين والمصادقة، وتشمل إنشاء الحسابات الجديدة، وتسجيل الدخول، وإدارة جلسات المستخدمين باستخدام رموز JWT.

وتعرض صفحة التوثيق لكل خدمة اسم نقطة النهاية (Endpoint)، وطريقة الإرسال (HTTP Method)، والمعاملات المطلوبة، ونوع كل معامل، بالإضافة إلى وصف مختصر لوظيفته.

وتسهم هذه المعلومات في تسهيل عملية دمج خدمات المصادقة داخل التطبيقات المختلفة، مع تقليل الأخطاء الناتجة عن إرسال بيانات غير صحيحة.

POST POST /api/auth/register

تسجيل حساب جديد لحفظ السجلات.

المعاملات

المعامل	النوع	مطلوب	الوصف
username	string	نعم ✓	اسم المستخدم
email	string	نعم ✓	البريد الإلكتروني
password	string	نعم ✓	كلمة المرور

POST POST /api/auth/login

تسجيل الدخول والحصول على JWT token.

المعاملات

المعامل	النوع	مطلوب	الوصف
email	string	نعم ✓	البريد الإلكتروني
password	string	نعم ✓	كلمة المرور

الشكل 4-9: توثيق واجهات التسجيل وتسجيل الدخول

يتضح من الشكل (4-9) أن النظام يقدم توثيقاً منظماً لواجهات المصادقة، حيث تُعرض جميع المعاملات المطلوبة مع نوع البيانات الخاصة بها، الأمر الذي يسهل على المطورين بناء تطبيقات تعتمد على خدمات النظام بصورة صحيحة.

3-4-4 دعم تعدد اللغات

يدعم النظام واجهة استخدام متعددة اللغات، حيث يمكن للمستخدم التبديل بين اللغة العربية واللغة الإنجليزية بصورة مباشرة دون الحاجة إلى إعادة تحميل الصفحة.

ولا يقتصر هذا الدعم على ترجمة النصوص فقط، بل يشمل أيضاً تغيير اتجاه الصفحة بين RTL و LTR، بالإضافة إلى تعديل اتجاه محرر النصوص وأزرار التحكم بما يتوافق مع اللغة المختارة، مما يضمن تجربة استخدام متكاملة للمستخدمين من مختلف اللغات.

تعدد اللغات

الموقع يدعم لغتين: العربية (افتراضي) و English.

التبديل: استخدم القائمة المنسدلة  أعلى الصفحة. اللغة المحفوظة في المتصفح.

الميزات حسب اللغة

- تبديل كامل لكل النصوص (عناوين، أزرار، مساعدات)
- تبديل اتجاه الصفحة (LTR ↔ RTL) تلقائياً
- تبديل اتجاه محرر النص
- تبديل أزرار الاتجاه (RTL/LTR) لتتطابق اللغة

الشكل 4-10: واجهة دعم تعدد اللغات

يتضح من الشكل (4-10) أن النظام يتيح التبديل الفوري بين اللغتين العربية والإنجليزية، مع تعديل اتجاه عناصر الواجهة تلقائياً، الأمر الذي يوفر تجربة استخدام متناسقة تتوافق مع طبيعة كل لغة.

4-4-4 دعم OpenClaw Skill

يتضمن النظام ملف OpenClaw Skill الذي يسمح بدمج خدمة التحويل مع وكلاء الذكاء الاصطناعي (AI Agents) والأدوات الداعمة لبروتوكول Model Context Protocol (MCP).

ويحتوي ملف المهارة على وصف كامل لإمكانات النظام، والوظائف التي يقدمها، بالإضافة إلى المعاملات المطلوبة وآلية استدعاء خدمة التحويل، مما يسمح للأدوات الخارجية باستخدام النظام بصورة مباشرة لتنفيذ عمليات تحويل المستندات.

OpenClaw Skill

MD-2-PDF متاح كـ **AgentSkill** لأي ذكاء اصطناعي يدعم OpenClaw (Claude, Gemini, Codex, Cursor). وأكثر).

الملف: SKILL.md في المستودع

الوصف: تحويل Markdown إلى PDF مع دعم RTL والعربية

المميزات الموثقة بال-Skill

الميزة	الوصف
تحويل سريع	Markdown → PDF عبر API أو MCP
RTL وعربي	دعم كامل للاتجاه من اليمين لليسار
LaTeX	معادلات رياضية سطرية وعرضية
تلوين الأكواد	GitHub Dark Dimmed theme
PlantUML	مخططات مباشرة في المستند
ثيمات	5 ثيمات: أزرق، أحمر، أخضر، بنفسجي، ذهبي
MCP Server	5 أدوات للتكامل المباشر

الشكل 4-11: توثيق OpenClaw Skill

يتضح من الشكل (4-11) أن النظام يوفر ملف مهارة (Skill) يصف جميع الوظائف المدعومة، بما في ذلك تحويل Markdown إلى PDF، ودعم اللغة العربية، والمعادلات الرياضية، وتلوين النصوص البرمجية، ومخططات PlantUML، الأمر الذي يسهل دمج النظام مع بيئات الذكاء الاصطناعي الحديثة.

4-5 دعم بروتوكول (MCP) Model Context Protocol

حرص المشروع على توفير واجهة متوافقة مع بروتوكول (MCP) Model Context Protocol، والذي أصبح معياراً حديثاً لربط تطبيقات الذكاء الاصطناعي بالأدوات والخدمات الخارجية. ويتيح هذا البروتوكول للوكلاء الذكيين تنفيذ عمليات التحويل وإدارة المستندات من خلال استدعاءات قياسية وآمنة، دون الحاجة إلى تطوير واجهات خاصة لكل تطبيق.

وقد تم تنفيذ خادم MCP ليعمل مباشرة فوق منصة Cloudflare Pages باستخدام بروتوكول Streamable HTTP، مما يسمح بالوصول إلى خدمات النظام عبر نقطة نهاية موحدة، مع الاستفادة من مزايا البنية السحابية الطرفية مثل انخفاض زمن الاستجابة وقابلية التوسع التلقائي.

🤖 خادم MCP (عن بُعد على Cloudflare)

نقطة النهاية الوحيدة: <https://md-2-pdf.pages.dev/mcp>

خادم MCP يعمل مباشرة على Cloudflare Pages عبر **Streamable HTTP** — لا حاجة لتثبيت Node.js أو استنساخ المشروع أو بنائه محلياً. أي عميل MCP متوافق يتصل بهذا الرابط فقط.

الخادم يوفر 5 أدوات للذكاء الاصطناعي لتحويل Markdown والتحكم بسجل التحويلات:

convert-markdown-to-pdf تحويل مباشر

تحويل نص Markdown إلى ملف PDF (Base64). يدعم: frontmatter، تلوين الكود، رياضيات KaTeX، رسوم PlantUML، تخطيطات عربية/RTL.

المعامل	النوع	الوصف
markdown	string	نص Markdown (مطلوب)
pageSize	enum	A4 Letter Legal
orientation	enum	portrait landscape
direction	enum	rtl ltr hybrid
fontSize	number	حجم الخط (8-24)
css	string	CSS إضافي
margin	object	{ top, right, bottom, left }
filename	string	اسم الملف المقترح

الشكل 4-12: الواجهة الرئيسية لخادم MCP

يتضح من الشكل (4-12) أن خادم MCP يوفر نقطة نهاية موحدة يمكن لأي عميل متوافق مع البروتوكول استخدامها للوصول إلى خدمات النظام، كما يعرض مجموعة الأدوات المتاحة ومعاملات كل أداة، مما يسهل عملية التكامل مع تطبيقات الذكاء الاصطناعي المختلفة.

4-5-1 أدوات خادم MCP

يوفر خادم MCP مجموعة من الأدوات التي تغطي الوظائف الرئيسة للنظام، وتشمل تحويل مستندات Markdown إلى PDF، واستعراض المستندات المحفوظة، وجلب مستند محدد، بالإضافة إلى إنشاء جلسات مؤقتة للمستخدمين الضيوف. ويعرض التوثيق لكل أداة اسمها، ووظيفتها، والمعاملات المطلوبة، وأنواع البيانات المقبولة، مما يساعد المطورين على استدعاء الأدوات بصورة صحيحة داخل تطبيقاتهم.

list-documents سجل

عرض المستندات المحولة سابقاً للمستخدم أو الجلسة الضيف. يتطلب auth.

المعامل	النوع	الوصف
limit	number	عدد النتائج (1-200)
offset	number	ترقيم الصفحات
search	string	بحث في العناوين

get-document سجل

جلب مستند واحد بالكامل (Markdown + metadata) عبر معرفه الرقمي.

المعامل	النوع	الوصف
id	number	معرف المستند (مطلوب)

create-guest-session auth

إنشاء جلسة ضيف UUID لحفظ التحويلات بدون تسجيل حساب. استخدم الناتج في meta.auth.guestSessionId_.

الشكل 4-13: أدوات خادم MCP

يتضح من الشكل (4-13) أن النظام يوفر مجموعة متكاملة من الأدوات القابلة للاستدعاء عبر بروتوكول MCP، حيث تشمل أدوات التحويل، وإدارة المستندات، وإنشاء جلسات الضيوف، مما يعزز قابلية النظام للتكامل مع تطبيقات الذكاء الاصطناعي والأنظمة الخارجية.

4-6 نتائج التطبيق واختبار وظائف النظام

بعد الانتهاء من تنفيذ النظام وتطوير جميع مكوناته، تم إجراء مجموعة من الاختبارات العملية للتحقق من قدرة النظام على معالجة مختلف عناصر مستندات Markdown وإنتاج ملفات PDF مطابقة للمحتوى الأصلي. وشملت الاختبارات جميع الخصائص الرئيسية التي يدعمها النظام، مثل التنسيقات الأساسية، ودعم اللغة العربية، والمعادلات الرياضية، وتلوين النصوص البرمجية، ومخططات PlantUML.

وقد أظهرت نتائج الاختبارات نجاح النظام في تنفيذ جميع الوظائف المطلوبة، مع المحافظة على جودة التنسيق ودقة عرض العناصر المختلفة داخل المستند النهائي.

4-6-1 التحقق من خصائص النظام

قبل تنفيذ اختبارات التحويل التفصيلية، تم التحقق من تفعيل جميع الخصائص التي يوفرها النظام، مثل دعم اللغة العربية، والمعاينة المباشرة، وتصدير ملفات PDF، ودعم المعادلات الرياضية، وتلوين النصوص البرمجية، بالإضافة إلى معالجة مخططات PlantUML.

وقد أظهرت نتائج الاختبار أن جميع هذه الخصائص تعمل بصورة صحيحة دون الحاجة إلى إعدادات إضافية من قبل المستخدم.

محول Markdown إلى PDF

أداة احترافية لتحويل Markdown إلى PDF مع دعم كامل للغة العربية.

المميزات

الميزة	الحالة
دعم RTL	مُدعوم ✓
المعاينة	مُدعومة ✓
التصدير PDF	مُدعوم ✓
LaTeX	مُدعوم ✓
أكواد البرمجة	مُدعومة ✓
PlantUML	مُدعوم ✓

الشكل 4-14: التحقق من خصائص النظام المدعومة

يتضح من الشكل (4-14) أن جميع الوظائف الرئيسة للنظام تعمل بنجاح، حيث تظهر حالة جميع الخصائص على أنها مدعومة، بما يشمل دعم اتجاه الكتابة RTL، والمعاينة الفورية، وتصدير ملفات PDF، والمعادلات الرياضية، وتلوين النصوص البرمجية، ومخططات PlantUML، مما يؤكد جاهزية النظام للاستخدام العملي.

4-6-2 اختبار التنسيق الأساسية

تم اختبار قدرة النظام على معالجة عناصر Markdown الأساسية، والتي تشمل العناوين، والنصوص العريضة والمائلة، والقوائم المرتبة وغير المرتبة، والروابط التشعبية، بالإضافة إلى عناصر التنسيق الأخرى.

وأظهرت النتائج أن النظام استطاع تحويل جميع هذه العناصر إلى مستند PDF مع المحافظة على تنسيقها الأصلي دون حدوث أي تشوهات في التخطيط أو اتجاه النصوص.

التنسيقات الأساسية

يمكنك كتابة نص عريض أو مائل أو كلاهما.

هذا نص مقتبس. يمكنك استخدامه لتأكيد نقطة هامة.

القوائم

1. عنصر مرقم أول
2. عنصر مرقم ثاني
 - عنصر فرعي
 - عنصر فرعي آخر

الروابط والصور

[اضغط هنا لزيارة الموقع](#)

الشكل 4-15: اختبار التنسيقات الأساسية

يتضح من الشكل (4-15) أن النظام يعرض عناصر Markdown الأساسية بصورة صحيحة داخل المستند النهائي، حيث تظهر العناوين والقوائم والروابط والتنسيقات المختلفة بالشكل المتوقع، مما يؤكد سلامة عملية التحويل من Markdown إلى PDF.

4-6-3 اختبار دعم المعادلات الرياضية (LaTeX)

لاختبار دعم المعادلات الرياضية، تم إنشاء مستند يحتوي على معادلات مكتوبة بصيغة LaTeX، تشمل معادلات سطرية وأخرى مستقلة، بالإضافة إلى مصفوفات وتعبيرات رياضية معقدة.

واعتمد النظام على مكتبة KaTeX لتفسير هذه المعادلات وتحويلها إلى عناصر HTML قبل إدراجها داخل ملف PDF.

▲ معادلات رياضية (LaTeX)

معادلة أينشتاين: $E = mc^2$

معادلة عرضية:

$$\int_0^{\infty} e^{-x^2} dx = \frac{\sqrt{\pi}}{2}$$

مصفوفة:

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

الشكل 4-16: اختبار دعم المعادلات الرياضية

يتضح من الشكل (4-16) أن النظام نجح في عرض المعادلات الرياضية بدقة عالية، مع المحافظة على تنسيق الرموز الرياضية وأحجامها ومواضعها داخل المستند، الأمر الذي يجعله مناسباً لإعداد التقارير العلمية والوثائق الأكاديمية.

4-6-4 اختبار تلوين النصوص البرمجية

تم اختبار النظام باستخدام أمثلة لعدد من لغات البرمجة، مثل JavaScript وPython وCSS وSQL وJSON، للتحقق من كفاءة مكتبة Highlight.js في تمييز الكلمات المفتاحية والرموز البرمجية.

وأظهرت النتائج نجاح النظام في تطبيق ألوان مخصصة لكل لغة برمجية، مع المحافظة على تنسيق الكتل البرمجية داخل ملف PDF.

أكواد البرمجة

JavaScript

JAVASCRIPT

```
function fibonacci(n) {
  if (n ≤ 1) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}
console.log(fibonacci(10)); // 55
```

Python

PYTHON

```
def factorial(n):
  if n ≤ 1:
    return 1
  return n * factorial(n - 1)

print(factorial(5)) # 120
```

CSS

CSS

```
.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 20px;
  direction: rtl;
}
```

الشكل 4-17: اختبار تلوين النصوص البرمجية

يتضح من الشكل (4-17) أن النظام يدعم عدداً كبيراً من لغات البرمجة، ويعرض النصوص البرمجية داخل كتل برمجية منسقة مع تطبيق تمييز نحوي (Syntax Highlighting)، مما يسهل قراءة النصوص البرمجية داخل المستندات التقنية.

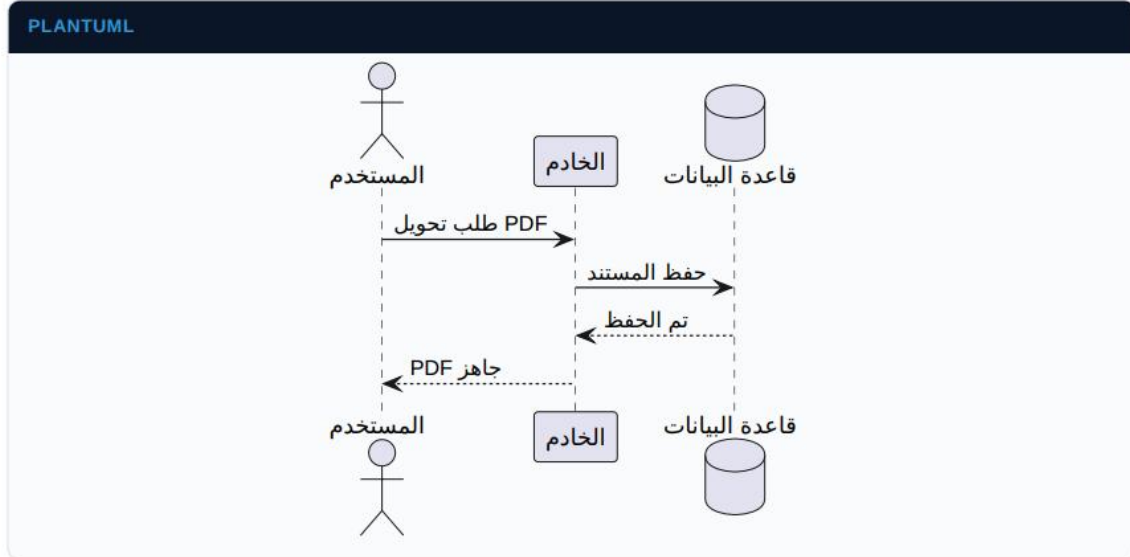
4-6-5 اختبار مخططات PlantUML

تم اختبار دعم مخططات PlantUML باستخدام عدة أنواع من المخططات، بهدف التحقق من قدرة النظام على تحويلها إلى رسومات متجهة SVG ودمجها داخل المستند النهائي دون فقدان الجودة.

وشملت الاختبارات ثلاثة أنواع رئيسة من المخططات، وهي المخططات التسلسلية، ومخططات الفئات، ومخططات التدفق.

مخططات PlantUML

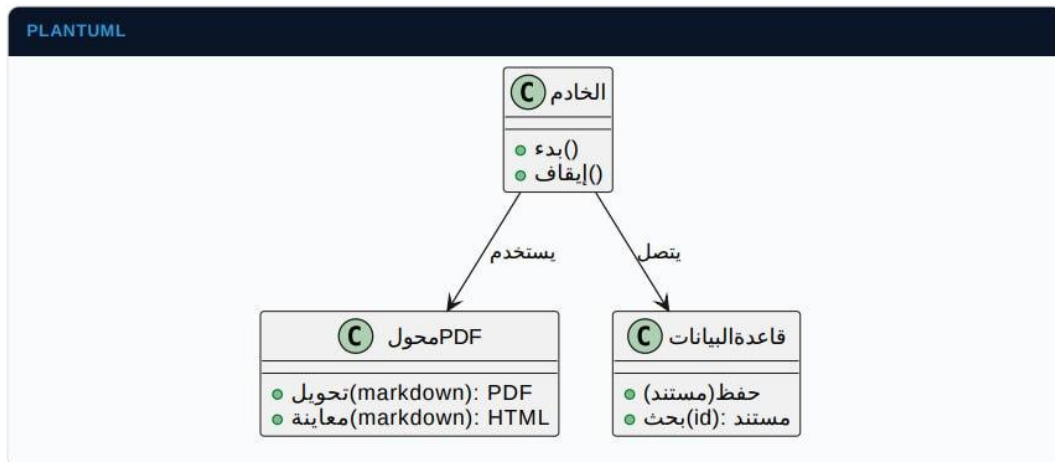
مخطط تسلسلي



الشكل 4-18: اختبار المخطط التسلسلي (Sequence Diagram)

يتضح من الشكل (4-18) أن النظام نجح في تحويل المخطط التسلسلي إلى رسم متجهي عالي الجودة، مع المحافظة على جميع العناصر والعلاقات بين الكائنات المشاركة في عملية التفاعل.

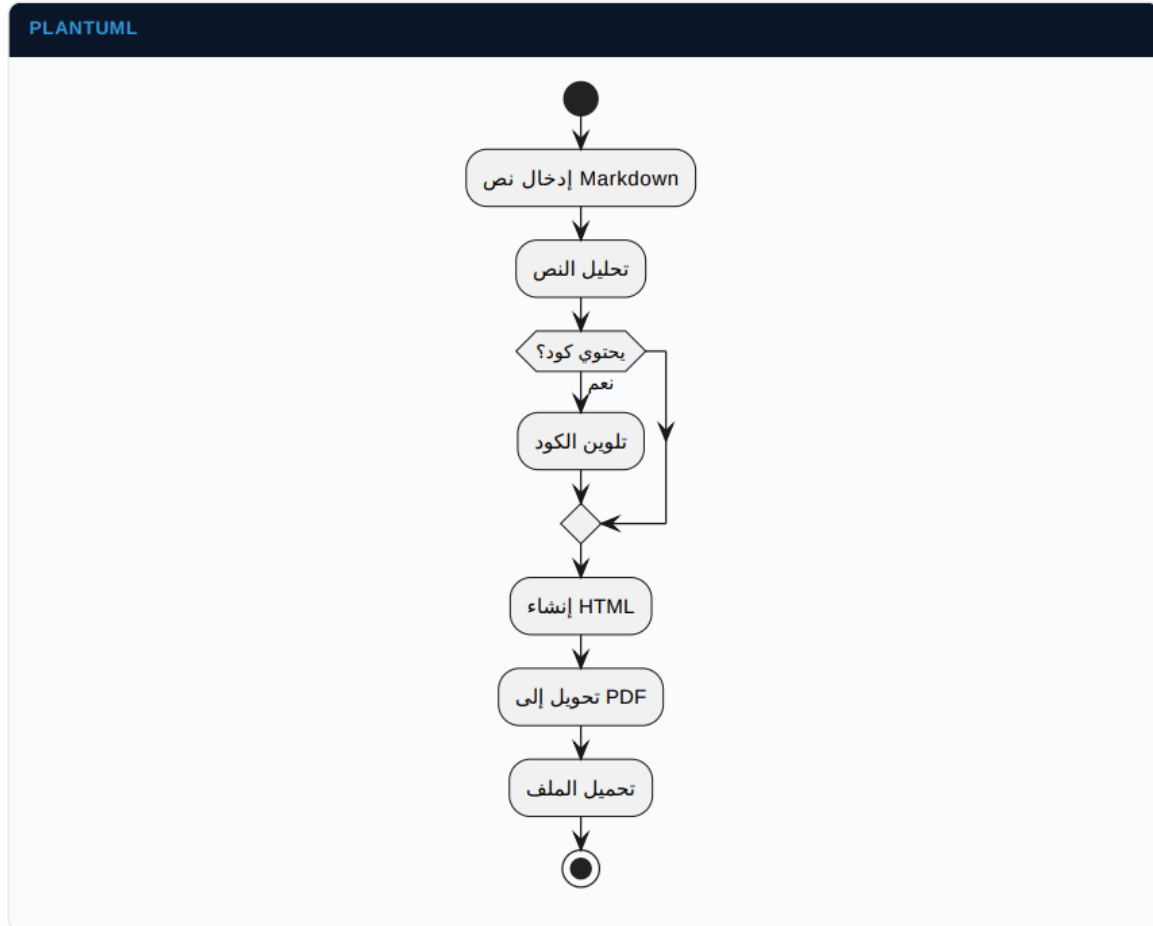
مخطط فئات



الشكل 4-19: اختبار مخطط الفئات (Class Diagram)

يتضح من الشكل (4-19) أن النظام يعرض مخطط الفئات بصورة واضحة، مع الحفاظ على العلاقات بين الأصناف والخصائص والعمليات، مما يؤكد نجاح معالجة مخططات UML داخل المستند النهائي.

مخطط تدفق



الشكل 4-20: اختبار مخطط التدفق (Activity / Flow Diagram)

يتضح من الشكل (4-20) أن النظام استطاع تحويل مخطط التدفق إلى رسم متجهي عالي الدقة مع المحافظة على جميع العقد والمسارات واتجاهات الانتقال، وهو ما يثبت نجاح آلية دمج مخططات PlantUML داخل ملفات PDF دون الحاجة إلى تحويلها إلى صور نقطية منخفضة الجودة.

الفصل الخامس: الخاتمة والآفاق

المستقبلية

يقدم هذا المشروع حلاً متكاملاً لتحويل المستندات المكتوبة بلغة Markdown إلى ملفات PDF عالية الجودة من خلال بنية سحابية حديثة تعتمد على تقنيات Serverless والحوسبة الطرفية، مع توفير واجهة استخدام سهلة وواجهة برمجة تطبيقات تدعم التكامل مع الأنظمة الخارجية، إضافة إلى دعم بروتوكول Model Context Protocol الذي يتيح الاستفادة من خدمات النظام ضمن تطبيقات ووكلاء الذكاء الاصطناعي. وقد ركز المشروع على معالجة عدد من التحديات التي تعاني منها أدوات التحويل التقليدية، وفي مقدمتها دعم اللغة العربية واتجاه الكتابة من اليمين إلى اليسار، وعرض المعادلات الرياضية، وتلوين النصوص البرمجية، ودمج مخططات PlantUML بجودة عالية داخل المستند النهائي، مع المحافظة على دقة التنسيق وسهولة الاستخدام.

وأظهرت نتائج الاختبارات العملية نجاح النظام في تنفيذ جميع الوظائف المخططة بكفاءة، حيث تمكن من معالجة مختلف عناصر مستندات Markdown وإنتاج ملفات PDF احترافية مع الحفاظ على جودة العرض وسلامة التخطيط، كما أثبتت البنية المعتمدة قدرتها على تقديم أداء جيد وقابلية للتوسع دون الحاجة إلى إدارة خوادم تقليدية، وهو ما ينسجم مع الاتجاهات الحديثة في تطوير تطبيقات الويب والخدمات السحابية. كذلك أسهم توفير واجهات برمجة تطبيقات موثقة وخادم متوافق مع بروتوكول MCP في زيادة مرونة النظام وإمكانية دمجها ضمن تطبيقات متنوعة، الأمر الذي يجعله مناسباً للاستخدام في المجالات الأكاديمية والبحثية والتقنية.

ويمثل المشروع أساساً يمكن البناء عليه مستقبلاً من خلال إضافة مزيد من الخصائص التي تعزز قدراته وتوسع مجالات استخدامه، مثل دعم تنسيقات إدخال وإخراج إضافية، وتوفير قوالب احترافية قابلة للتخصيص، وإتاحة معالجة المستندات الكبيرة بصورة متوازية لتحسين الأداء، وإضافة إمكانيات التعاون بين المستخدمين وإدارة الإصدارات، إلى جانب دمج تقنيات الذكاء الاصطناعي لتقديم اقتراحات لتحسين بنية المستندات أو اكتشاف الأخطاء قبل التحويل. كما يمكن تطوير النظام ليدعم خدمات التخزين السحابي المختلفة، وإضافة آليات متقدمة لإدارة الصلاحيات وتتبع الاستخدام، وتوسيع دعم بروتوكول MCP ليتكامل مع عدد أكبر من المنصات والوكلاء الذكيين، مما يساهم في تعزيز قابلية النظام للتوسع ويجعله منصة متكاملة لإدارة وتحويل المستندات الرقمية مستقبلاً.

- [1] Cloudflare. (2022, July 15). Working in public: Our docs-as-code approach. Cloudflare Blog. Retrieved May 18, 2026, from <https://blog.cloudflare.com/our-docs-as-code-approach/>
- [2] Cloudflare. (2026). Cloudflare Browser Rendering Documentation. Retrieved June 10, 2026, from <https://developers.cloudflare.com/browser-rendering/>
- [3] Marin, E., Perino, D., & Di Pietro, R. (2022). Serverless computing: A security perspective. *Journal of Cloud Computing*, 11(69). <https://doi.org/10.1186/s13677-022-00347-w>
- [4] Unicode Consortium. (2025). Unicode Standard Annex #9: Unicode Bidirectional Algorithm. Retrieved June 7, 2026, from <https://www.unicode.org/reports/tr9/>
- [5] PlantUML. (2026). PlantUML Language Reference Guide. Retrieved June 5, 2026, from <https://plantuml.com/>
- [6] Cloudflare. (2026). Cloudflare Pages Documentation. Retrieved May 25, 2026, from <https://developers.cloudflare.com/pages/>
- [7] Cloudflare. (2026). Cloudflare Browser Rendering Documentation. Retrieved June 10, 2026, from <https://developers.cloudflare.com/browser-rendering/>
- [8] Cloudflare. (2026). Cloudflare D1 Documentation. Retrieved June 8, 2026, from <https://developers.cloudflare.com/d1/>
- [9] Hono Team. (2026). Hono Documentation. Retrieved May 27, 2026, from <https://hono.dev/>
- [10] Zod. (2026). Zod Documentation. Retrieved June 1, 2026, from <https://zod.dev/>
- [11] Auth0. (2024). JSON Web Token (JWT) Best Practices. Retrieved June 2, 2026, from <https://auth0.com/docs/secure/tokens/json-web-tokens>
- [12] Marked. (2026). Marked Documentation. Retrieved May 30, 2026, from <https://marked.js.org/>
- [13] Gray-matter. (2026). Gray-matter Documentation. Retrieved May 30, 2026, from <https://github.com/jonschlinkert/gray-matter>
- [14] Highlight.js. (2026). Highlight.js Documentation. Retrieved June 4, 2026, from <https://highlightjs.org/>
- [15] KaTeX Contributors. (2026). KaTeX Documentation. Retrieved June 5, 2026, from <https://katex.org/>

- [16] WHATWG. (2025). HTML Living Standard. Retrieved June 6, 2026, from <https://html.spec.whatwg.org/>
- [17] Ecma International. (2024). ECMAScript® 2024 Language Specification. Retrieved June 7, 2026, from <https://tc39.es/ecma262/>
- [18] Wen, J., Chen, Z., Jin, X., & Liu, X. (2022). Rise of the Planet of Serverless Computing: A Systematic Review. arXiv. <https://doi.org/10.48550/arXiv.2206.12275>

Syrian Arab Republic

Ministry of Higher Education and Scientific Research

Al-Wataniya Private University

Faculty of Engineering

Department of Computer Engineering



Graduation Project Submitted in Partial Fulfillment of the Requirements for the Bachelor's
Degree in Computer Engineering

Design and Implementation of an API and MCP Tool for Converting Markdown Documents to PDF

Prepared by:

Abdullah Mohammad Nawar Al-Shaqfa

Supervised by:

Prof. Dr. Ibrahim Al-Shami

Academic Year 2025 - 2026